

# 12. 스레드 풀과 Executor 프레임워크1

#1.인강/0.자바/5.자바-고급1편

- /스레드를 직접 사용할 때의 문제점
- /Executor 프레임워크 소개
- /ExecutorService 코드로 시작하기
- /Runnable의 불편함
- /Future1 - 시작
- /Future2 - 분석
- /Future3 - 활용
- /Future4 - 이유
- /Future5 - 정리
- /Future6 - 취소
- /Future7 - 예외
- /ExecutorService - 작업 컬렉션 처리
- /문제와 풀이
- /정리

## 스레드를 직접 사용할 때의 문제점

실무에서 스레드를 직접 생성해서 사용하면 다음과 같은 3가지 문제가 있다.

1. 스레드 생성 시간으로 인한 성능 문제
2. 스레드 관리 문제
3. Runnable 인터페이스의 불편함

### 1. 스레드 생성 비용으로 인한 성능 문제

스레드를 사용하려면 먼저 스레드를 생성해야 한다. 그런데 스레드는 다음과 같은 이유로 매우 무겁다.

- **메모리 할당:** 각 스레드는 자신만의 호출 스택(call stack)을 가지고 있어야 한다. 이 호출 스택은 스레드가 실행되는 동안 사용하는 메모리 공간이다. 따라서 스레드를 생성할 때는 이 호출 스택을 위한 메모리를 할당해야 한다.
- **운영체제 자원 사용:** 스레드를 생성하는 작업은 운영체제 커널 수준에서 이루어지며, 시스템 콜(system call)을 통해 처리된다. 이는 CPU와 메모리 리소스를 소모하는 작업이다.
- **운영체제 스케줄러 설정:** 새로운 스레드가 생성되면 운영체제의 스케줄러는 이 스레드를 관리하고 실행 순서를 조정해야 한다. 이는 운영체제의 스케줄링 알고리즘에 따라 추가적인 오버헤드가 발생할 수 있다.
- 참고로 스레드 하나는 보통 1MB 이상의 메모리를 사용한다.

스레드를 생성하는 작업은 상대적으로 무겁다. 단순히 자바 객체를 하나 생성하는 것과는 비교할 수 없을 정도로 큰 작업이다.

예를 들어서 어떤 작업 하나를 수행할 때 마다 스레드를 각각 생성하고 실행한다면, 스레드의 생성 비용 때문에, 이미 많은 시간이 소모된다. 아주 가벼운 작업이라면, 작업의 실행 시간보다 스레드의 생성 시간이 더 오래 걸릴 수도 있다.

이런 문제를 해결하려면 생성한 스레드를 재사용하는 방법을 고려할 수 있다. 스레드를 재사용하면 처음 생성할 때를 제외하고는 생성을 위한 시간이 들지 않는다. 따라서 스레드가 아주 빠르게 작업을 수행할 수 있다.

## 2. 스레드 관리 문제

서버의 CPU, 메모리 자원은 한정되어 있기 때문에, 스레드는 무한하게 만들 수 없다.

예를 들어서, 사용자의 주문을 처리하는 서비스라고 가정하자. 그리고 사용자의 주문이 들어올 때 마다 스레드를 만들어서 요청을 처리한다고 가정하겠다. 서비스 마케팅을 위해 선착순 할인 이벤트를 진행한다고 가정해보자. 그러면 사용자가 갑자기 몰려들 수 있다. 평소 동시에 100개 정도의 스레드면 충분했는데, 갑자기 10000개의 스레드가 필요한 상황이 된다면 CPU, 메모리 자원이 버티지 못할 것이다.

이런 문제를 해결하려면 우리 시스템이 버틸 수 있는, 최대 스레드의 수 까지만 스레드를 생성할 수 있게 관리해야 한다.

또한 이런 문제도 있다. 예를 들어 애플리케이션을 종료한다고 가정해보자.

이때 안전한 종료를 위해 실행 중인 스레드가 남은 작업은 모두 수행한 다음에 프로그램을 종료하고 싶거나, 또는 급하게 종료해야 해서 인터럽트 등의 신호를 주고 스레드를 종료하고 싶다고 가정해보자.

이런 경우에도 스레드가 어딘가에 관리가 되어 있어야 한다.

## 3. Runnable 인터페이스의 불편함

```
public interface Runnable {  
    void run();  
}
```

Runnable 인터페이스는 다음과 같은 이유로 불편하다.

- **반환 값이 없다:** `run()` 메서드는 반환 값을 가지지 않는다. 따라서 실행 결과를 얻기 위해서는 별도의 메커니즘을 사용해야 한다. 쉽게 이야기해서 스레드의 실행 결과를 직접 받을 수 없다. 앞에서 공부한 `SumTask`의 예를 생각해보자. 스레드가 실행한 결과를 멤버 변수에 넣어두고, `join()` 등을 사용해서 스레드가 종료되길 기다린 다음에 멤버 변수에 보관한 값을 받아야 한다.
- **예외 처리:** `run()` 메서드는 체크 예외(`checked exception`)를 던질 수 없다. 체크 예외의 처리는 메서드 내부에서 처리해야 한다.

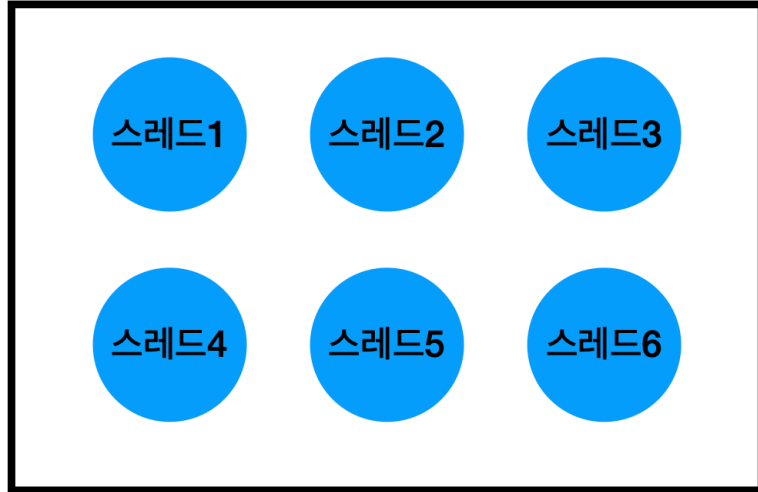
이런 문제를 해결하려면 반환 값도 받을 수 있고, 예외도 좀 더 쉽게 처리할 수 있는 방법이 필요하다. 추가로 반환 값 뿐만 아니라 해당 스레드에서 발생한 예외도 받을 수 있다면 더 좋을 것이다.

## 해결

지금까지 설명한 1번, 2번 문제를 해결하려면 스레드를 생성하고 관리하는 풀(Pool)이 필요하다.

### 1. 작업 요청

task1



스레드 풀(Thread Pool)

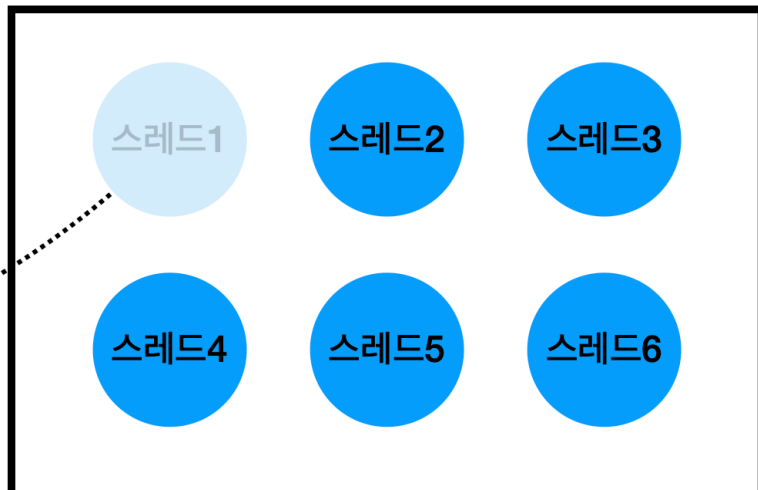
- 스레드를 관리하는 스레드 풀(스레드가 모여서 대기하는 수영장 풀 같은 개념)에 스레드를 미리 필요한 만큼 만들어둔다.
- 스레드는 스레드 풀에서 대기하며 쉰다.
- 작업 요청이 온다.

### 3. 작업 처리

task1

스레드1

2. 풀에서 스레드 조회



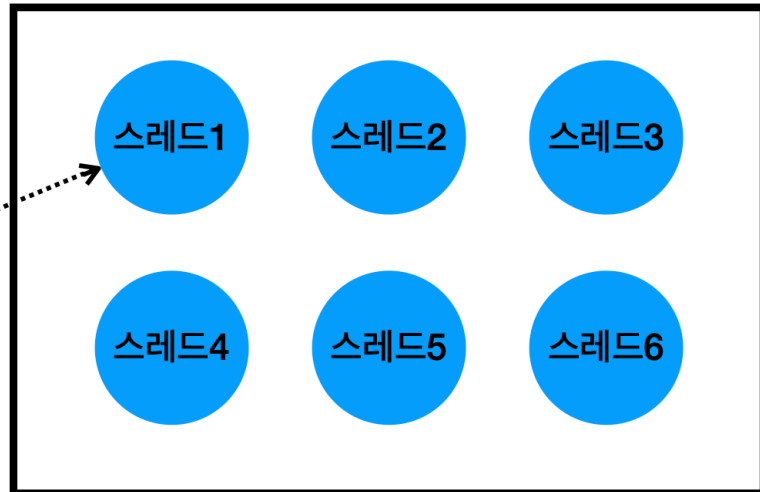
스레드 풀(Thread Pool)

- 스레드 풀에서 이미 만들어진 스레드를 하나 조회한다.
- 조회한 스레드1로 작업을 처리한다.

#### 4. 작업 완료



#### 5. 풀에 스레드 반납



### 스레드 풀(Thread Pool)

- 스레드1은 작업을 완료한다.
- 작업을 완료한 스레드는 종료하는게 아니라, 다시 스레드 풀에 반납한다. 스레드1은 이후에 다시 재사용 될 수 있다.

이렇게 스레드 풀이라는 개념을 사용하면 스레드를 재사용할 수 있어서, 재사용시 스레드의 생성 시간을 절약할 수 있다. 그리고 스레드 풀에서 스레드가 관리되기 때문에 필요한 만큼만 스레드를 만들 수 있고, 또 관리할 수 있다.

사실 스레드 풀이라는 것이 별것이 아니다. 그냥 컬렉션에 스레드를 보관하고 재사용할 수 있게 하면 된다. 하지만 스레드 풀에 있는 스레드는 처리할 작업이 없다면, 대기(WAITING) 상태로 관리해야 하고, 작업 요청이 오면 RUNNABLE 상태로 변경해야 한다. 막상 구현하려고 하면 생각보다 매우 복잡하다는 사실을 알게될 것이다. 여기에 생산자 소비자 문제까지 겹친다. 잘 생각해보면 어떤 생산자가 작업(task)을 만들 것이고, 우리의 스레드 풀에 있는 스레드가 소비자가 되는 것이다.

이런 문제를 한방에 해결해주는 것이 바로 자바가 제공하는 Executor 프레임워크다.

Executor 프레임워크는 스레드 풀, 스레드 관리, Runnable의 문제점은 물론이고, 생산자 소비자 문제까지 한방에 해결해주는 자바 멀티스레드 최고의 도구이다. 지금까지 우리가 배운 멀티스레드 기술의 총 집합이 여기에 들어있다.

참고로 앞서 설명한 이유와 같이 스레드를 사용할 때는 생각보다 고려해야 할 일이 많다. 그래서 실무에서는 스레드를 직접 하나하나 생성해서 사용하는 일이 드물다. 대신에 지금부터 설명할 Executor 프레임워크를 주로 사용하는데, 이 기술을 사용하면 매우 편리하게 멀티스레드 프로그래밍을 할 수 있다.

## Executor 프레임워크 소개

자바의 Executor 프레임워크는 멀티스레딩 및 병렬 처리를 쉽게 사용할 수 있도록 돕는 기능의 모음이다. 이 프레임워크는 작업 실행의 관리 및 스레드 풀 관리를 효율적으로 처리해서 개발자가 직접 스레드를 생성하고 관리하는 복잡함을 줄여준다.

## Executor 프레임워크의 주요 구성 요소

### Executor 인터페이스

```
package java.util.concurrent;

public interface Executor {
    void execute(Runnable command);
}
```

- 가장 단순한 작업 실행 인터페이스로, `execute(Runnable command)` 메서드 하나를 가지고 있다.

### ExecutorService 인터페이스 - 주요 메서드

```
public interface ExecutorService extends Executor, AutoCloseable {

    <T> Future<T> submit(Callable<T> task);

    @Override
    default void close(){...}

    ...
}
```

- `Executor` 인터페이스를 확장해서 작업 제출과 제어 기능을 추가로 제공한다.
- 주요 메서드로는 `submit()`, `close()` 가 있다.
- 더 많은 기능이 있지만 나머지 기능들은 뒤에서 알아보자.
- `Executor` 프레임워크를 사용할 때는 대부분 이 인터페이스를 사용한다.

`ExecutorService` 인터페이스의 기본 구현체는 `ThreadPoolExecutor` 이다.

우선은 이런 것이 있구나 정도만 보면 된다. 직접 코드로 실행하면서 학습해보자.

## 로그 출력 유틸리티 만들기

먼저 `Executor` 프레임워크의 상태를 확인하기 위한 로그 출력 유틸리티를 만들어두자.

```

package thread.executor;

import java.util.concurrent.ExecutorService;
import java.util.concurrent.ThreadPoolExecutor;

import static util.MyLogger.log;

public abstract class ExecutorUtils {

    public static void printState(ExecutorService executorService) {
        if (executorService instanceof ThreadPoolExecutor poolExecutor) {
            int pool = poolExecutor.getPoolSize();
            int active = poolExecutor.getActiveCount();
            int queuedTasks = poolExecutor.getQueue().size();
            long completedTask = poolExecutor.getCompletedTaskCount();
            log("[pool=" + pool + ", active=" + active + ", queuedTasks=" +
queuedTasks + ", completedTasks=" + completedTask + "]");
        } else {
            log(executorService);
        }
    }
}

```

- pool: 스레드 풀에서 관리되는 스레드의 숫자
- active: 작업을 수행하는 스레드의 숫자
- queuedTasks: 큐에 대기중인 작업의 숫자
- completedTask: 완료된 작업의 숫자

참고로 `ExecutorService` 인터페이스는 `getPoolSize()`, `getActiveCount()` 같은 자세한 기능은 제공하지 않는다. 이 기능은 `ExecutorService`의 대표 구현체인 `ThreadPoolExecutor`를 사용해야 한다.

`printState()` 메서드에 `ThreadPoolExecutor` 구현체가 넘어오면 우리가 구성한 로그를 출력하고, 그렇지 않은 경우에는 인스턴스 자체를 출력한다.

## ExecutorService 코드로 시작하기

먼저 1초간 대기하는 아주 간단한 작업을 하나 만들자.

```

package thread.executor;

import static util.MyLogger.log;
import static util.ThreadUtils.sleep;

public class RunnableTask implements Runnable {
    private final String name;
    private int sleepMs = 1000;

    public RunnableTask(String name) {
        this.name = name;
    }

    public RunnableTask(String name, int sleepMs) {
        this.name = name;
        this.sleepMs = sleepMs;
    }

    @Override
    public void run() {
        log(name + " 시작");
        sleep(sleepMs); // 작업 시간 시뮬레이션
        log(name + " 완료");
    }
}

```

- `Runnable` 인터페이스를 구현한다. 1초의 작업이 걸리는 간단한 작업으로 가정하자.

```

package thread.executor;

import java.util.concurrent.ExecutorService;
import java.util.concurrent.LinkedBlockingQueue;
import java.util.concurrent.ThreadPoolExecutor;
import java.util.concurrent.TimeUnit;

import static thread.executor.ExecutorUtils.*;
import static util.MyLogger.log;
import static util.ThreadUtils.sleep;

public class ExecutorBasicMain {

    public static void main(String[] args) throws InterruptedException {

```

```

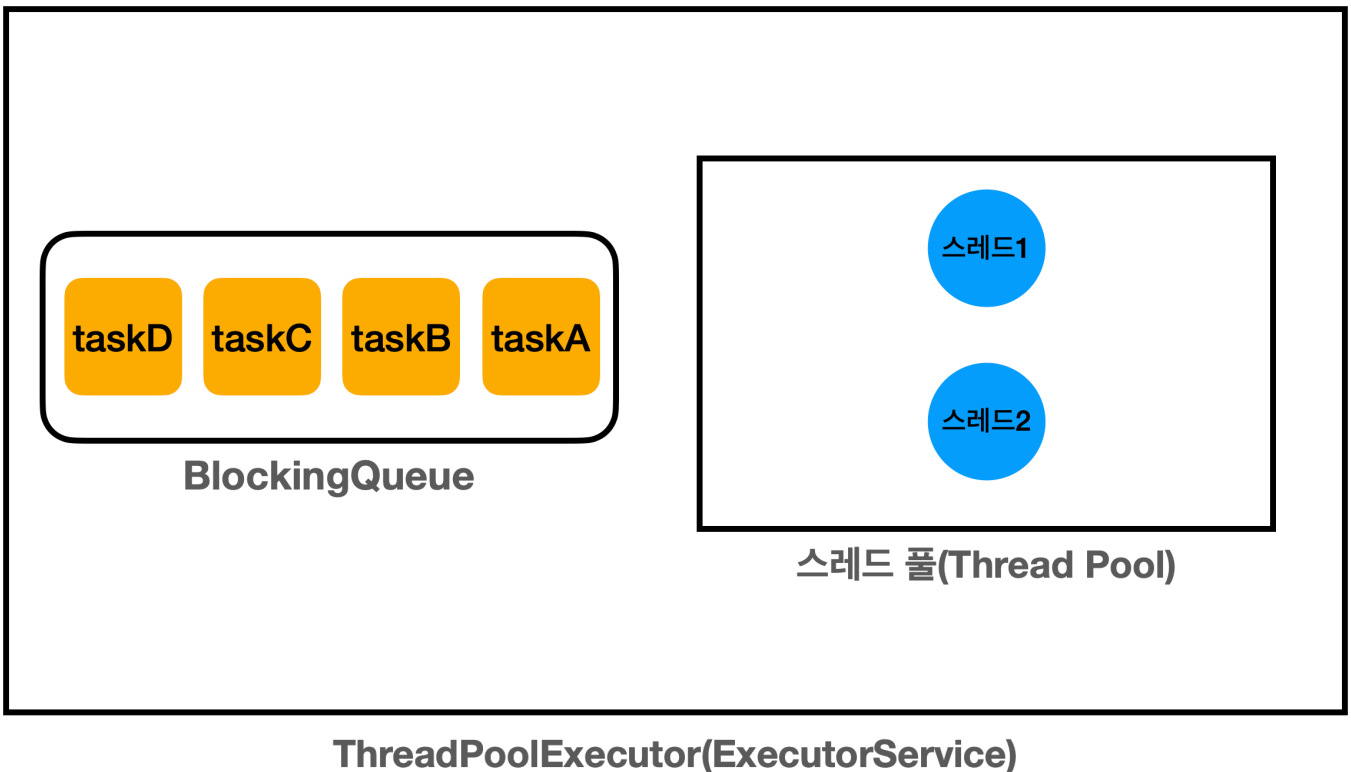
    ExecutorService es = new ThreadPoolExecutor(2, 2, 0,
    TimeUnit.MILLISECONDS, new LinkedBlockingQueue<>());
    log("== 초기 상태 ==");
    printState(es);
    es.execute(new RunnableTask("taskA"));
    es.execute(new RunnableTask("taskB"));
    es.execute(new RunnableTask("taskC"));
    es.execute(new RunnableTask("taskD"));
    log("== 작업 수행 중 ==");
    printState(es);

    sleep(3000);
    log("== 작업 수행 완료 ==");
    printState(es);

    es.close();
    log("== shutdown 완료 ==");
    printState(es);
}
}

```

ExecutorService의 가장 대표적인 구현체는 ThreadPoolExecutor이다.



ThreadPoolExecutor(ExecutorService)는 크게 2가지 요소로 구성되어 있다.

- 스레드 풀: 스레드를 관리한다.
- `BlockingQueue`: 작업을 보관한다. 생산자 소비자 문제를 해결하기 위해 단순한 큐가 아니라, `BlockingQueue`를 사용한다.

생산자가 `es.execute(new RunnableTask("taskA"))`를 호출하면, `RunnableTask("taskA")` 인스턴스가 `BlockingQueue`에 보관된다.

- **생산자**: `es.execute(작업)`를 호출하면 내부에서 `BlockingQueue`에 작업을 보관한다. `main` 스레드가 생산자가 된다.
- **소비자**: 스레드 풀에 있는 스레드가 소비자이다. 이후에 소비자 중에 하나가 `BlockingQueue`에 들어있는 작업을 받아서 처리한다.

## ThreadPoolExecutor 생성자

`ThreadPoolExecutor`의 생성자는 다음 속성을 사용한다.

- `corePoolSize`: 스레드 풀에서 관리되는 기본 스레드의 수
- `maximumPoolSize`: 스레드 풀에서 관리되는 최대 스레드 수
- `keepAliveTime, TimeUnit unit`: 기본 스레드 수를 초과해서 만들어진 스레드가 생존할 수 있는 대기 시간이다. 이 시간 동안 처리할 작업이 없다면 초과 스레드는 제거된다.
- `BlockingQueue workQueue`: 작업을 보관할 블로킹 큐

```
new ThreadPoolExecutor(2,2,0, TimeUnit.MILLISECONDS, new
LinkedBlockingQueue<>());
```

- 최대 스레드 수와 `keepAliveTime, TimeUnit unit`에 대한 부분은 뒤에서 따로 설명하겠다.
- 여기서는 `corePoolSize=2, maximumPoolSize=2`를 사용해서 기본 스레드와 최대 스레드 수를 맞추었다. 따라서 풀에서 관리되는 스레드는 2개로 고정된다. `keepAliveTime, TimeUnit unit`는 0으로 설정했는데, 이 부분은 뒤에서 설명한다.
- 작업을 보관할 블로킹 큐의 구현체로 `LinkedBlockingQueue`를 사용했다. 참고로 이 블로킹 큐는 작업을 무한대로 저장할 수 있다.

## 실행 결과

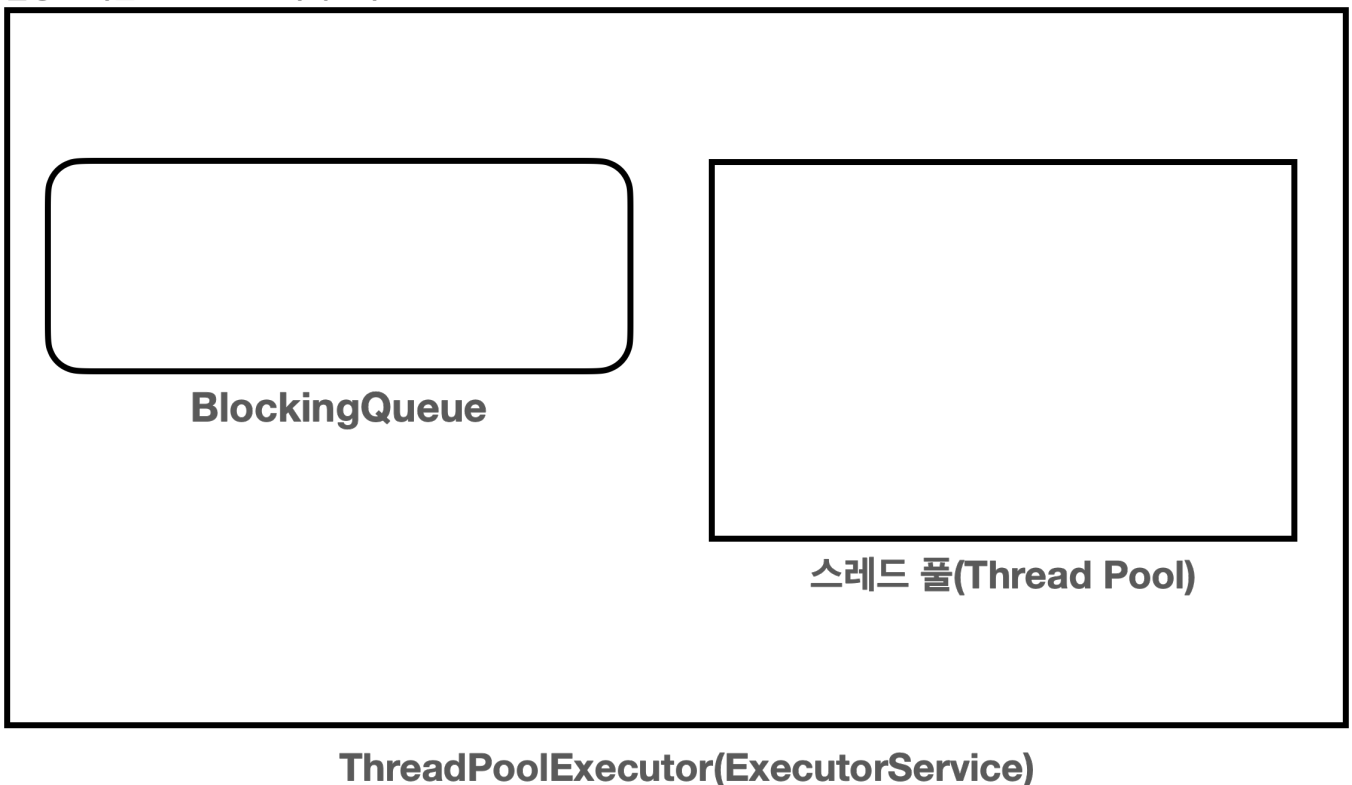
```
12:10:54.451 [    main] == 초기 상태 ==
12:10:54.461 [    main] [pool=0, active=0, queuedTasks=0, completedTasks=0]
12:10:54.461 [    main] == 작업 수행 중 ==
12:10:54.461 [    main] [pool=2, active=2, queuedTasks=2, completedTasks=0]
12:10:54.461 [pool-1-thread-1] taskA 시작
12:10:54.461 [pool-1-thread-2] taskB 시작
12:10:55.467 [pool-1-thread-1] taskA 완료
12:10:55.467 [pool-1-thread-2] taskB 완료
```

```

12:10:55.468 [pool-1-thread-1] taskC 시작
12:10:55.468 [pool-1-thread-2] taskD 시작
12:10:56.471 [pool-1-thread-2] taskD 완료
12:10:56.474 [pool-1-thread-1] taskC 완료
12:10:57.465 [    main] == 작업 수행 완료 ==
12:10:57.466 [    main] [pool=2, active=0, queuedTasks=0, completedTasks=4]
12:10:57.468 [    main] == shutdown 완료 ==
12:10:57.469 [    main] [pool=0, active=0, queuedTasks=0, completedTasks=4]

```

실행 순서를 그림으로 분석해보자.

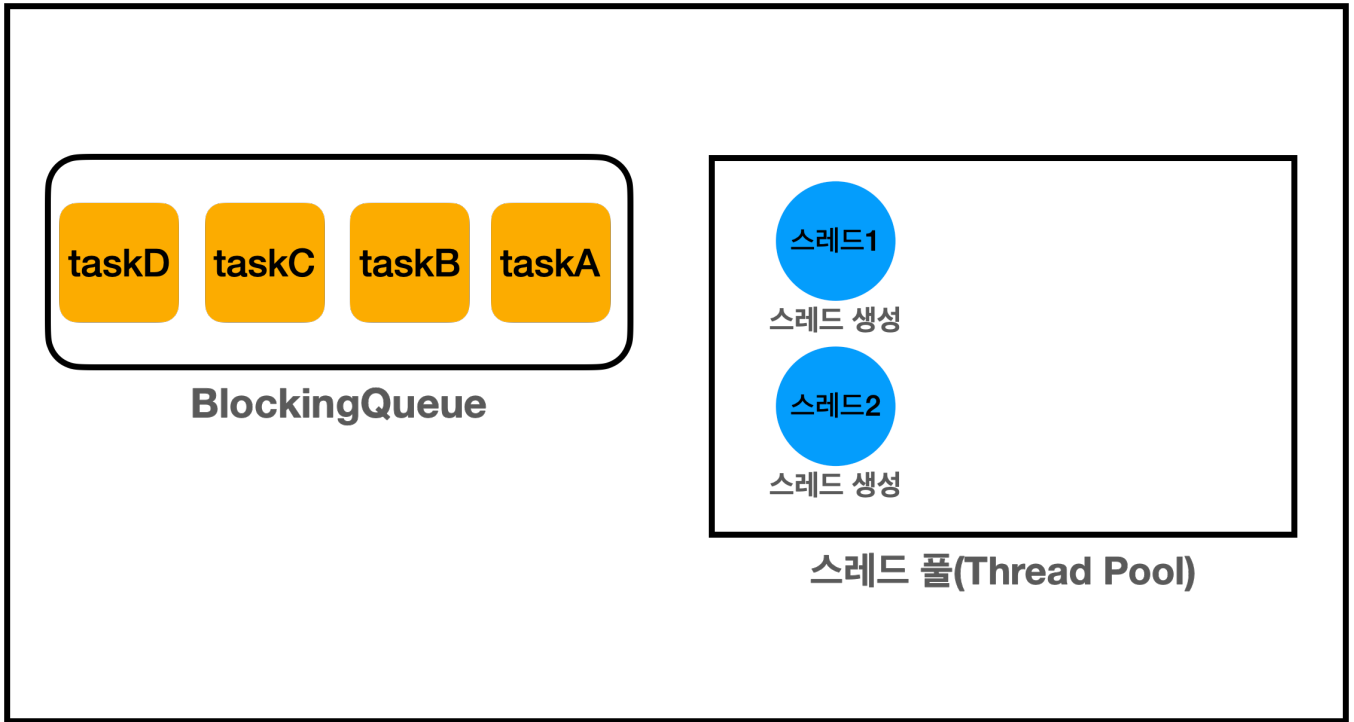


```

12:10:54.451 [    main] == 초기 상태 ==
12:10:54.461 [    main] [pool=0, active=0, queuedTasks=0, completedTasks=0]

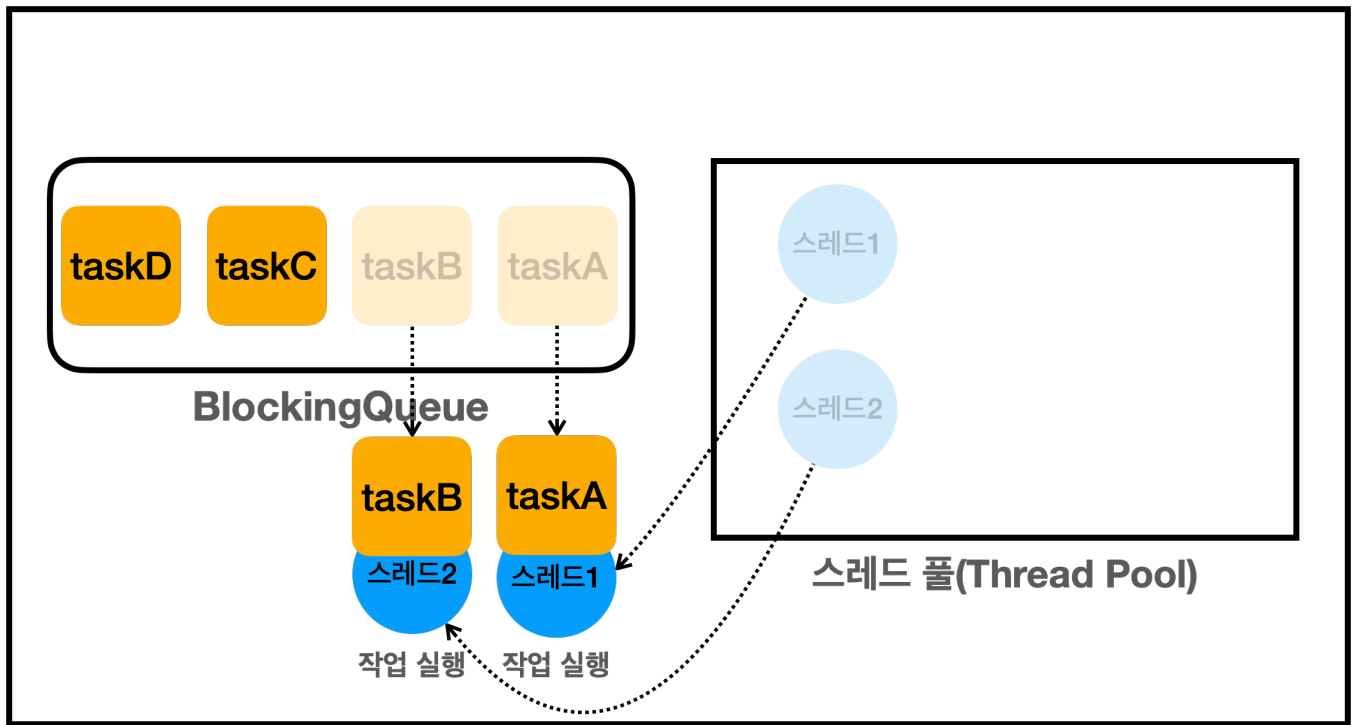
```

- `ThreadPoolExecutor`를 생성한 시점에 스레드 풀에 스레드를 미리 만들어두지는 않는다.



### ThreadPoolExecutor(ExecutorService)

- `main` 스레드가 `es.execute("taskA ~ taskD")` 를 호출한다.
  - 참고로 당연한 이야기지만 `main` 스레드는 작업을 전달하고 기다리지 않는다. 전달한 작업은 다른 스레드가 실행할 것이다. `main` 스레드는 작업을 큐에 보관까지만 하고 바로 다음 코드를 수행한다.
- `taskA~D` 요청이 블로킹 큐에 들어온다.
- 최초의 작업이 들어오면 이때 작업을 처리하기 위해 스레드를 만든다.
  - 참고로 스레드 풀에 스레드를 미리 만들어두지는 않는다.
- 작업이 들어올 때 마다 `corePoolSize` 의 크기 까지 스레드를 만든다.
  - 예를 들어서 최초 작업인 `taskA` 가 들어오는 시점에 스레드1을 생성하고, 다음 작업인 `taskB` 가 들어오는 시점에 스레드2를 생성한다.
  - 이런 방식으로 `corePoolSize` 에 지정한 수 만큼 스레드를 스레드 풀에 만든다. 여기서는 2를 설정했으므로 2개까지 만든다.
  - `corePoolSize` 까지 스레드가 생성되고 나면, 이후에는 스레드를 생성하지 않고 앞서 만든 스레드를 재사용한다.

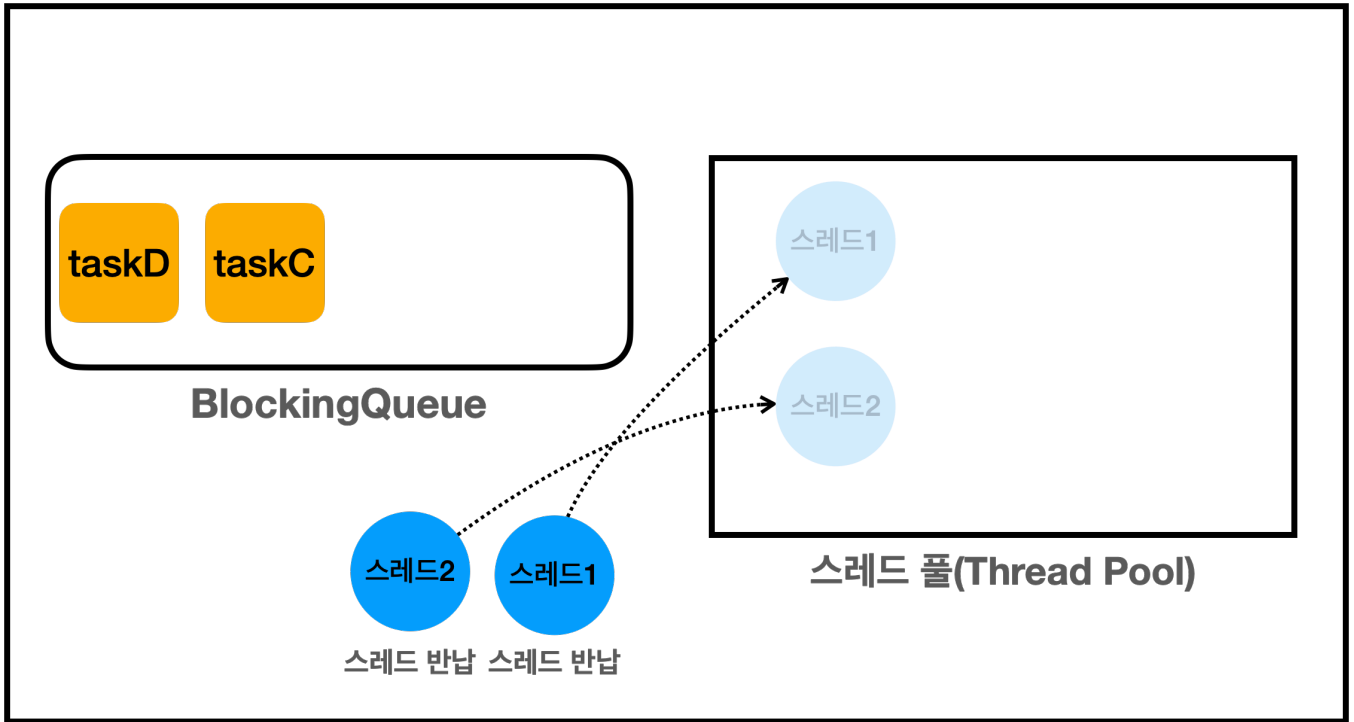


### ThreadPoolExecutor(ExecutorService)

```
12:10:54.461 [    main] == 작업 수행 중 ==
12:10:54.461 [    main] [pool=2, active=2, queuedTasks=2, completedTasks=0]
```

- 스레드 풀에 관리되는 스레드가 2개이므로 `pool=2`
- 작업을 수행중인 스레드가 2개이므로 `active=2`
- 큐에 대기중인 작업이 2개이므로 `queuedTasks=2`
- 완료된 작업은 없으므로 `completedTasks=0`

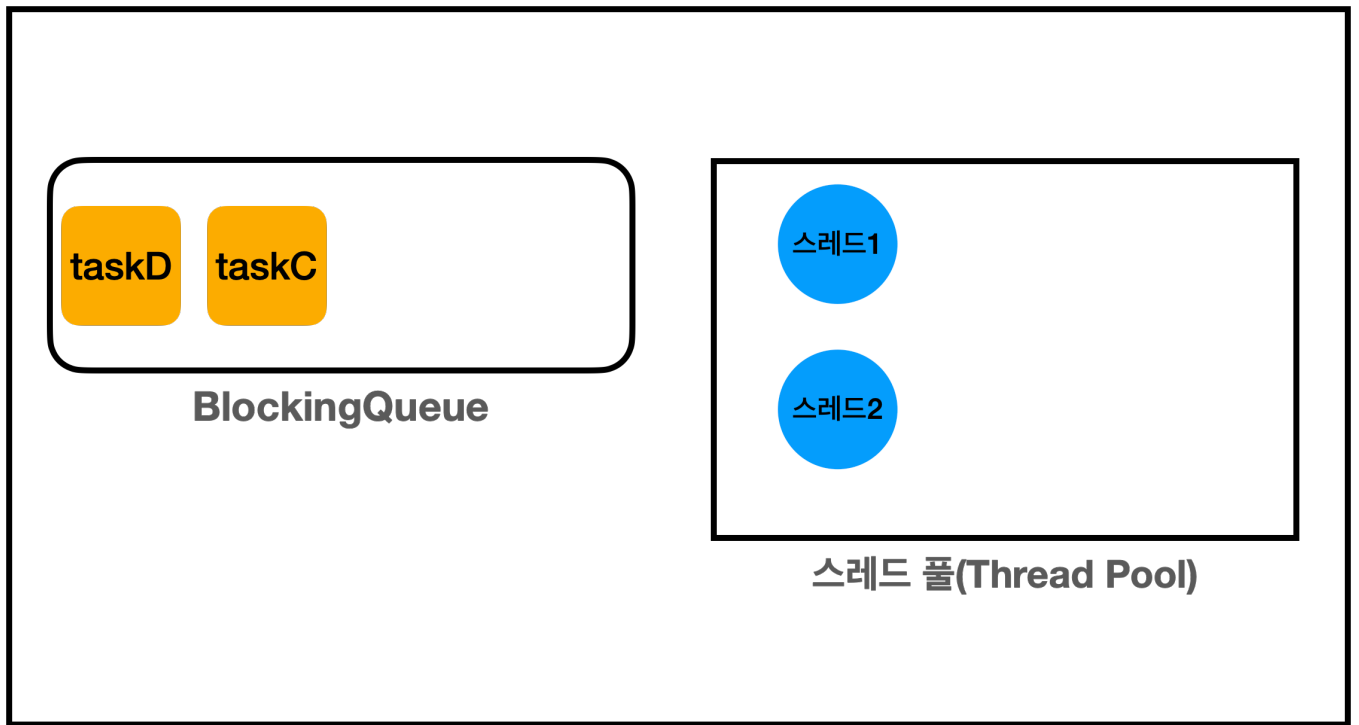
참고로 이해를 돕기 위해 스레드 풀의 스레드가 작업을 실행할 때, 그림으로는 스레드 풀에서 스레드를 꺼내는 것 처럼 표현했지만, 실제로 꺼내는 것은 아니고, 스레드의 상태가 변경된다고 이해하면 된다. 그래서 여전히 `pool=2`로 유지된다.



### ThreadPoolExecutor(ExecutorService)



- 작업이 완료되면 스레드 풀에 스레드를 반납한다. 스레드를 반납하면 스레드는 대기(WAITING) 상태로 스레드 풀에 대기한다.
  - 참고로 실제 반납 되는게 아니라, 스레드의 상태가 변경된다고 이해하면 된다.

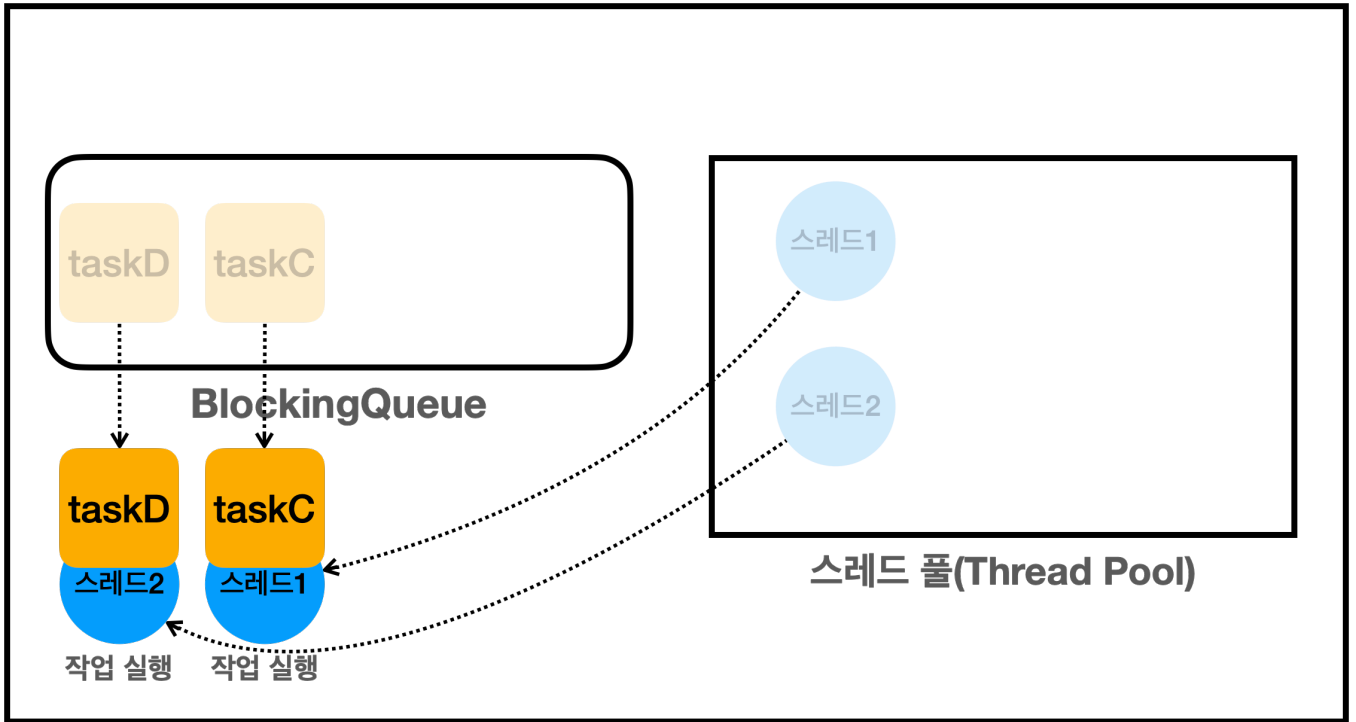


`ThreadPoolExecutor(ExecutorService)`



처리된 작업 처리된 작업

- 반납된 스레드는 재사용된다.

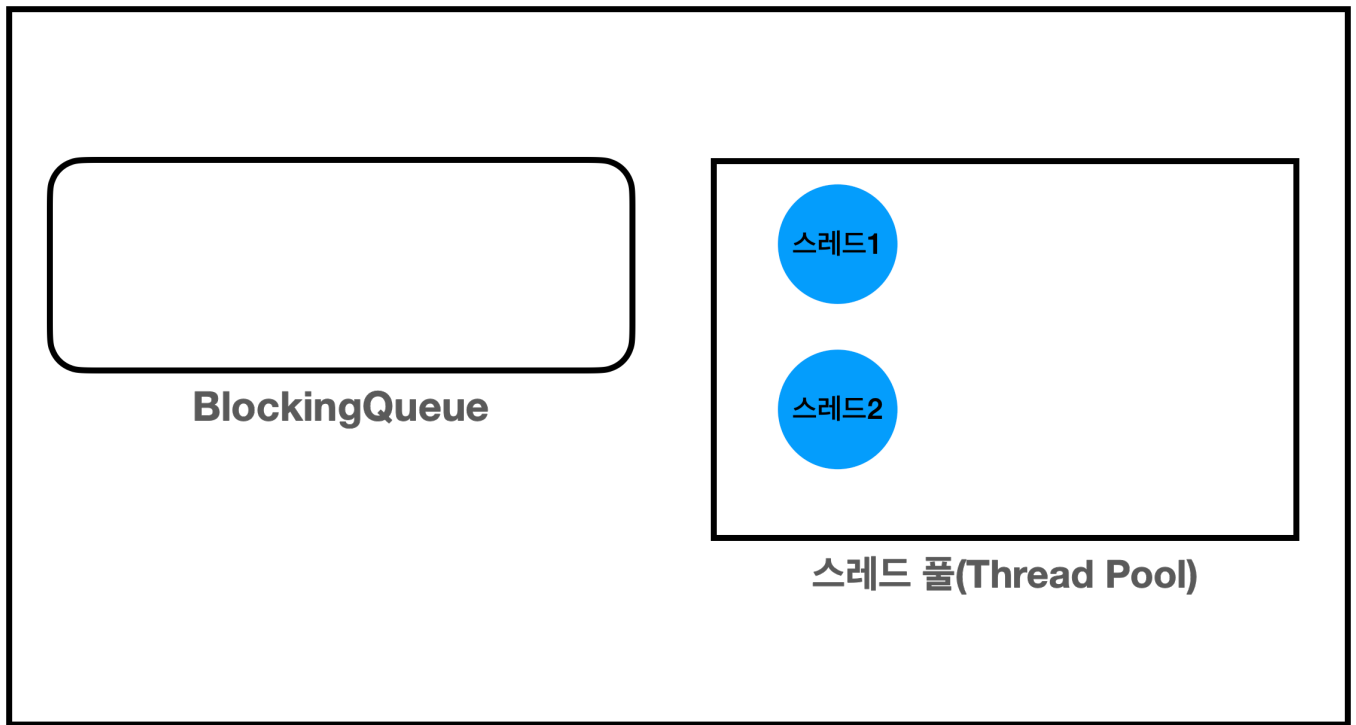


`ThreadPoolExecutor(ExecutorService)`



처리된 작업 처리된 작업

- `taskC`, `taskD`의 작업을 처리하기 위해 스레드 풀에서 스레드를 꺼내 재사용한다.



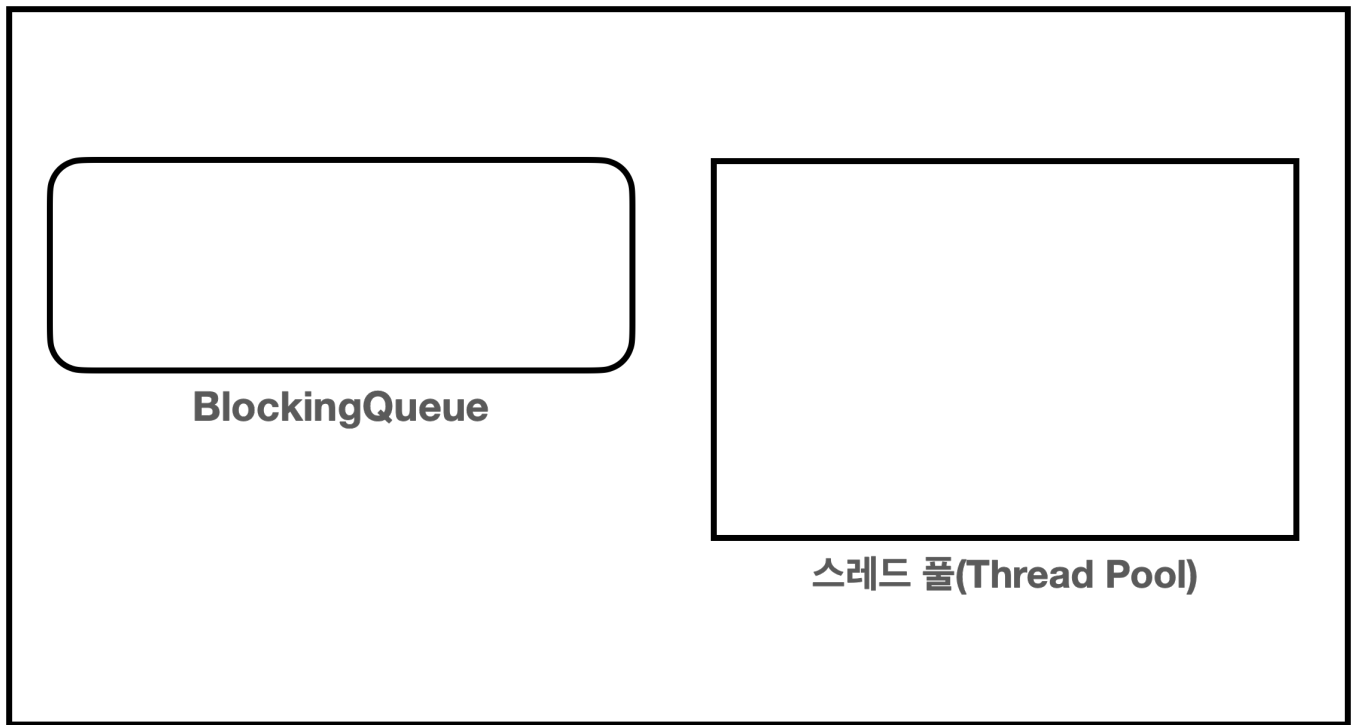
### ThreadPoolExecutor(ExecutorService)



처리된 작업 처리된 작업 처리된 작업 처리된 작업

- 작업이 완료되면 스레드는 다시 스레드 풀에서 대기한다.

```
12:10:57.465 [      main] == 작업 수행 완료 ==
12:10:57.466 [      main] [pool=2, active=0, queuedTasks=0, completedTasks=4]
```



### ThreadPoolExecutor(ExecutorService)



처리된 작업   처리된 작업   처리된 작업   처리된 작업

```
12:10:57.468 [      main] == shutdown 완료 ==
12:10:57.469 [      main] [pool=0, active=0, queuedTasks=0, completedTasks=4]
```

`close()` 를 호출하면 `ThreadPoolExecutor` 가 종료된다. 이때 스레드 풀에 대기하는 스레드도 함께 제거된다.

**참고:** `close()` 는 자바 19부터 지원되는 메서드이다. 만약 19 미만 버전을 사용한다면 `shutdown()` 을 호출하자. 둘의 차이는 뒤에서 설명한다.

## Runnable의 불편함

앞서 `Runnable` 인터페이스는 다음과 같은 불편함이 있다고 설명했다.

### Runnable 인터페이스의 불편함

```
public interface Runnable {  
    void run();  
}
```

- **반환 값이 없다:** `run()` 메서드는 반환 값을 가지지 않는다. 따라서 실행 결과를 얻기 위해서는 별도의 메커니즘을 사용해야 한다. 쉽게 이야기해서 스레드의 실행 결과를 직접 받을 수 없다. 앞에서 공부한 `SumTask`의 예를 생각해보자. 스레드가 실행한 결과를 멤버 변수에 넣어두고, `join()` 등을 사용해서 스레드가 종료될길 기다린 다음에 멤버 변수를 통해 값을 받아야 한다.
- **예외 처리:** `run()` 메서드는 체크 예외(`checked exception`)를 던질 수 없다. 체크 예외의 처리는 메서드 내부에서 처리해야 한다.

Executor 프레임워크는 어떤 방식으로 이런 불편함을 해결하는지 알아보자.

## Runnable 사용

이해를 돕기 위해 먼저 `Runnable`을 통해 별도의 스레드에서 무작위 값을 하나 구하는 간단한 코드를 작성해보자.

```
package thread.executor.future;  
  
import java.util.Random;  
  
import static util.MyLogger.log;  
import static util.ThreadUtils.sleep;  
  
public class RunnableMain {  
  
    public static void main(String[] args) throws InterruptedException {  
        MyRunnable task = new MyRunnable();  
        Thread thread = new Thread(task, "Thread-1");  
        thread.start();  
        thread.join();  
        int result = task.value;  
        log("result value = " + result);  
    }  
  
    static class MyRunnable implements Runnable {  
        int value;  
  
        @Override  
        public void run() {  
            log("Runnable 시작");  
            sleep(2000);  
        }  
    }  
}
```

```

        value = new Random().nextInt(10);
        log("create value = " + value);
        log("Runnable 완료");
    }
}

```

- **run()**: 0 ~ 9 사이의 무작위 값을 조회한다. 작업에 2초가 걸린다고 가정한다.

## 실행 결과

```

14:22:06.675 [ Thread-1] Runnable 시작
14:22:08.685 [ Thread-1] create value = 1
14:22:08.686 [ Thread-1] Runnable 완료
14:22:08.686 [      main] result value = 1

```

- 무작위 값이므로 숫자의 결과는 다를 수 있다.
- 프로그램이 시작되면 Thread-1이라는 별도의 스레드를 하나 만든다.
- Thread-1이 수행하는 MyRunnable은 무작위 값을 하나 구한 다음에 value 필드에 보관한다.
- 클라이언트인 main 스레드가 이 별도의 스레드에서 만든 무작위 값을 얻어오려면 Thread-1 스레드가 종료될 때까지 기다려야 한다. 그래서 main 스레드는 join()을 호출해서 대기한다.
- 이후에 main 스레드에서 MyRunnable 인스턴스의 value 필드를 통해 최종 무작위 값을 획득한다.

별도의 스레드에서 만든 무작위 값 하나를 받아오는 과정이 이렇게 복잡하다.

작업 스레드(Thread-1)는 값을 어딘가에 보관해두어야 하고, 요청 스레드(main)는 작업 스레드의 작업이 끝날 때까지 join()을 호출해서 대기한 다음에, 어딘가에 보관된 값을 찾아서 꺼내야 한다.

작업 스레드는 간단히 값을 return을 통해 반환하고, 요청 스레드는 그 반환 값을 바로 받을 수 있다면 코드가 훨씬 더 간결해질 것이다.

이런 문제를 해결하기 위해 Executor 프레임워크는 Callable과 Future라는 인터페이스를 도입했다.

## Future1 - 시작

### Runnable과 Callable 비교

Runnable은 다음과 같다.

```
package java.lang;

public interface Runnable {
    void run();
}
```

- Runnable의 run() 은 반환 타입이 void 이다. 따라서 값을 반환할 수 없다.
- 예외가 선언되어 있지 않다. 따라서 해당 인터페이스를 구현하는 모든 메서드는 체크 예외를 던질 수 없다.
  - 참고로 자식은 부모의 예외 범위를 넘어설 수 없다. 부모에 예외가 선언되어 있지 않으므로 예외를 던질 수 없다.
  - 물론 런타임(비체크)예외는 제외다.

Callable은 다음과 같다.

```
package java.util.concurrent;

public interface Callable<V> {
    V call() throws Exception;
}
```

- java.util.concurrent 에서 제공되는 기능이다.
- Callable의 call() 은 반환 타입이 제네릭 V 이다. 따라서 값을 반환할 수 있다.
- throws Exception 예외가 선언되어 있다. 따라서 해당 인터페이스를 구현하는 모든 메서드는 체크 예외인 Exception 과 그 하위 예외를 모두 던질 수 있다.

Callable 을 실제 어떻게 사용하는지 알아보자.

## Callable과 Future 사용

```
package thread.executor.future;

import java.util.Random;
import java.util.concurrent.*;

import static util.MyLogger.log;
import static util.ThreadUtils.sleep;

public class CallableMainV1 {
```

```

    public static void main(String[] args) throws ExecutionException,
        InterruptedException {
        ExecutorService es = Executors.newFixedThreadPool(1);
        Future<Integer> future = es.submit(new MyCallable());
        Integer result = future.get();
        log("result value = " + result);
        es.close();
    }

    static class MyCallable implements Callable<Integer> {
        @Override
        public Integer call() {
            log("Callable 시작");
            sleep(2000);
            int value = new Random().nextInt(10);
            log("create value = " + value);
            log("Callable 완료");
            return value;
        }
    }
}

```

java.util.concurrent.Executors가 제공하는 newFixedThreadPool(size) 을 사용하면 편리하게 ExecutorService를 생성할 수 있다.

## 기존 코드

```

ExecutorService es = new ThreadPoolExecutor(1,1,0, TimeUnit.MILLISECONDS, new
    LinkedBlockingQueue<>());

```

## 편의 코드

```

ExecutorService es = Executors.newFixedThreadPool(1);

```

## 실행 결과

```

14:39:47.764 [pool-1-thread-1] Callable 시작
14:39:49.776 [pool-1-thread-1] create value = 4
14:39:49.776 [pool-1-thread-1] Callable 완료

```

```
14:39:49.777 [      main] result value = 4
```

먼저 `MyCallable` 을 구현하는 부분을 보자.

- 숫자를 반환하므로 반환할 제네릭 타입을 `<Integer>` 로 선언했다.
- 구현은 `Runnable` 코드와 비슷한데, 유일한 차이는 결과를 필드에 담아두는 것이 아니라, 결과를 반환한다는 점이다. 따라서 결과를 보관할 별도의 필드를 만들지 않아도 된다.

## `submit()`

```
<T> Future<T> submit(Callable<T> task); //인터페이스 정의
```

`ExecutorService` 가 제공하는 `submit()` 을 통해 `Callable` 을 작업으로 전달할 수 있다.

```
Future<Integer> future = es.submit(new MyCallable());
```

`MyCallable` 인스턴스가 블로킹 큐에 전달되고, 스레드 풀의 스레드 중 하나가 이 작업을 실행할 것이다.

이때 작업의 처리 결과는 직접 반환되는 것이 아니라 `Future` 라는 특별한 인터페이스를 통해 반환된다.

```
Integer result = future.get();
```

`future.get()` 을 호출하면 `MyCallable` 의 `call()` 이 반환한 결과를 받을 수 있다.

참고로 `Future.get()` 은 `InterruptedException`, `ExecutionException` 체크 예외를 던진다. 여기서는 잡지말고 간단하게 밖으로 던지자. 예외에 대한 부분은 뒤에서 설명한다.

## Executor 프레임워크의 강점

요청 스레드가 결과를 받아야 하는 상황이라면, `Callable` 을 사용한 방식은 `Runnable` 을 사용하는 방식보다 훨씬 편리하다. 코드만 보면 복잡한 멀티스레드를 사용한다는 느낌보다는, 단순한 싱글 스레드 방식으로 개발한다는 느낌이 들 것이다.

이 과정에서 내가 스레드를 생성하거나, `join()` 으로 스레드를 제어하거나 한 코드는 전혀 없다. 심지어 `Thread` 라는 코드도 없다.

단순하게 `ExecutorService` 에 필요한 작업을 요청하고 결과를 받아서 쓰면 된다!

복잡한 멀티스레드를 매우 편리하게 사용할 수 있는 것이 바로 Executor 프레임워크의 큰 강점이다.

하지만 편리한 것은 편리한 것이고, 기반 원리를 제대로 이해해야 문제없이 사용할 수 있다.

여기서 잘 생각해보면 한 가지 애매한 상황이 있다.

`future.get()` 을 호출하는 요청 스레드(main)는 `future.get()` 을 호출 했을 때 2가지 상황으로 나뉘게 된다.

- `MyCallable` 작업을 처리하는 스레드 풀의 스레드가 작업을 완료했다.
- `MyCallable` 작업을 처리하는 스레드 풀의 스레드가 아직 작업을 완료하지 못했다.

`future.get()` 을 호출했을 때 스레드 풀의 스레드가 작업을 완료했다면 반환 받을 결과가 있을 것이다. 그런데 아직 작업을 처리중이라면 어떻게 될까?

이런 의문도 들 것이다. 왜 결과를 바로 반환하지 않고, 불편하게 `Future` 라는 객체를 대신 반환할까? 이 부분을 제대로 이해해야 한다.

## Future2 - 분석

`Future` 는 번역하면 미래라는 뜻이고, 여기서는 미래의 결과를 받을 수 있는 객체라는 뜻이다. 그렇다면 누구의 미래의 결과를 말하는 것일까? 다음 코드를 보자?

```
Future<Integer> future = es.submit(new MyCallable());
```

- `submit()` 의 호출로 `MyCallable` 의 인스턴스를 전달한다.
- 이때 `submit()` 은 `MyCallable.call()` 이 반환하는 무작위 숫자 대신에 `Future` 를 반환한다.
- 생각해보면 `MyCallable` 이 즉시 실행되어서 즉시 결과를 반환하는 것은 불가능하다. 왜냐하면 `MyCallable` 은 즉시 실행되는 것이 아니다. 스레드 풀의 스레드가 미래의 어떤 시점에 이 코드를 대신 실행해야 한다.
- `MyCallable.call()` 메서드는 호출 스레드가 실행하는 것도 아니고, 스레드 풀의 다른 스레드가 실행하기 때문에 언제 실행이 완료되어서 결과를 반환할 지 알 수 없다.
- 따라서 결과를 즉시 받는 것은 불가능하다. 이런 이유로 `es.submit()` 은 `MyCallable` 의 결과를 반환하는 대신에 `MyCallable` 의 결과를 나중에 받을 수 있는 `Future` 라는 객체를 대신 제공한다.
- 정리하면 `Future` 는 전달한 작업의 미래이다. 이 객체를 통해 전달한 작업의 미래 결과를 받을 수 있다.

단순하게 정리하면, `Future` 는 전달한 작업의 미래 결과를 담고 있다고 생각하면 된다.

이제 본격적으로 `Future` 가 어떻게 작동하는지 알아보자.

`CallableMainV2` 는 `CallableMainV1` 과 같은 코드에 로그를 추가했다.

```

package thread.executor.future;

import java.util.Random;
import java.util.concurrent.*;

import static util.MyLogger.log;
import static util.ThreadUtils.sleep;

public class CallableMainV2 {

    public static void main(String[] args) throws ExecutionException,
    InterruptedException {
        ExecutorService es = Executors.newFixedThreadPool(1);
        log("submit() 호출");
        Future<Integer> future = es.submit(new MyCallable());
        log("future 즉시 반환, future = " + future);

        log("future.get() [블로킹] 메서드 호출 시작 -> main 스레드 WAITING");
        Integer result = future.get();
        log("future.get() [블로킹] 메서드 호출 완료 -> , main 스레드 RUNNABLE");

        log("result value = " + result);
        log("future 완료, future = " + future);
        es.close();
    }

    static class MyCallable implements Callable<Integer> {
        @Override
        public Integer call() {
            log("Callable 시작");
            sleep(2000);
            int value = new Random().nextInt(10);
            log("create value = " + value);
            log("Callable 완료");
            return value;
        }
    }
}

```

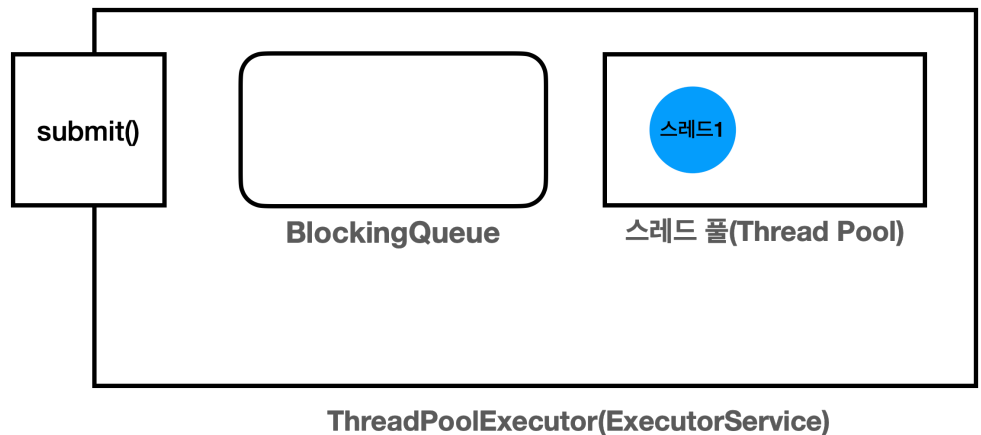
실행 결과

```

09:24:42.689 [      main] submit() 호출
09:24:42.691 [pool-1-thread-1] Callable 시작
09:24:42.691 [      main] future 즉시 반환, future = FutureTask@46d56d67[Not
completed, task = thread.executor.future.CallableMainV2$MyCallable@14acaea5]
09:24:42.691 [      main] future.get() [블로킹] 메서드 호출 시작 -> main 스레드 WAITING
09:24:44.703 [pool-1-thread-1] create value = 4
09:24:44.703 [pool-1-thread-1] Callable 완료
09:24:44.703 [      main] future.get() [블로킹] 메서드 호출 완료 -> , main 스레드
RUNNABLE
09:24:44.704 [      main] result value = 4
09:24:44.704 [      main] future 완료, future = FutureTask@46d56d67[Completed
normally]

```

## 실행 결과 분석



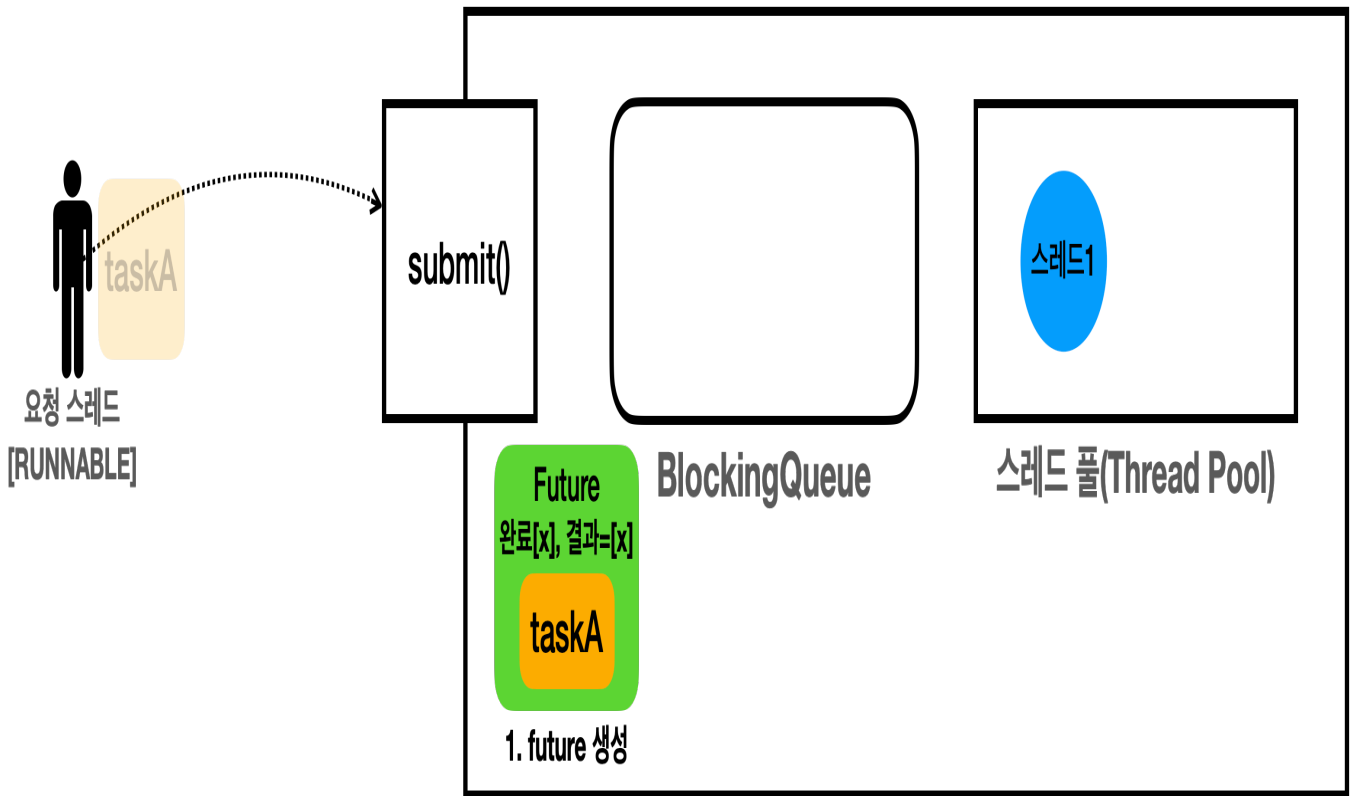
- MyCallable 인스턴스를 편의상 taskA 라고 하겠다.
- 편의상 스레드풀에 스레드가 1개 있다고 가정하겠다.

```
es.submit(new MyCallable())
```

```
09:24:42.689 [      main] submit() 호출, [논블로킹] 메서드
```

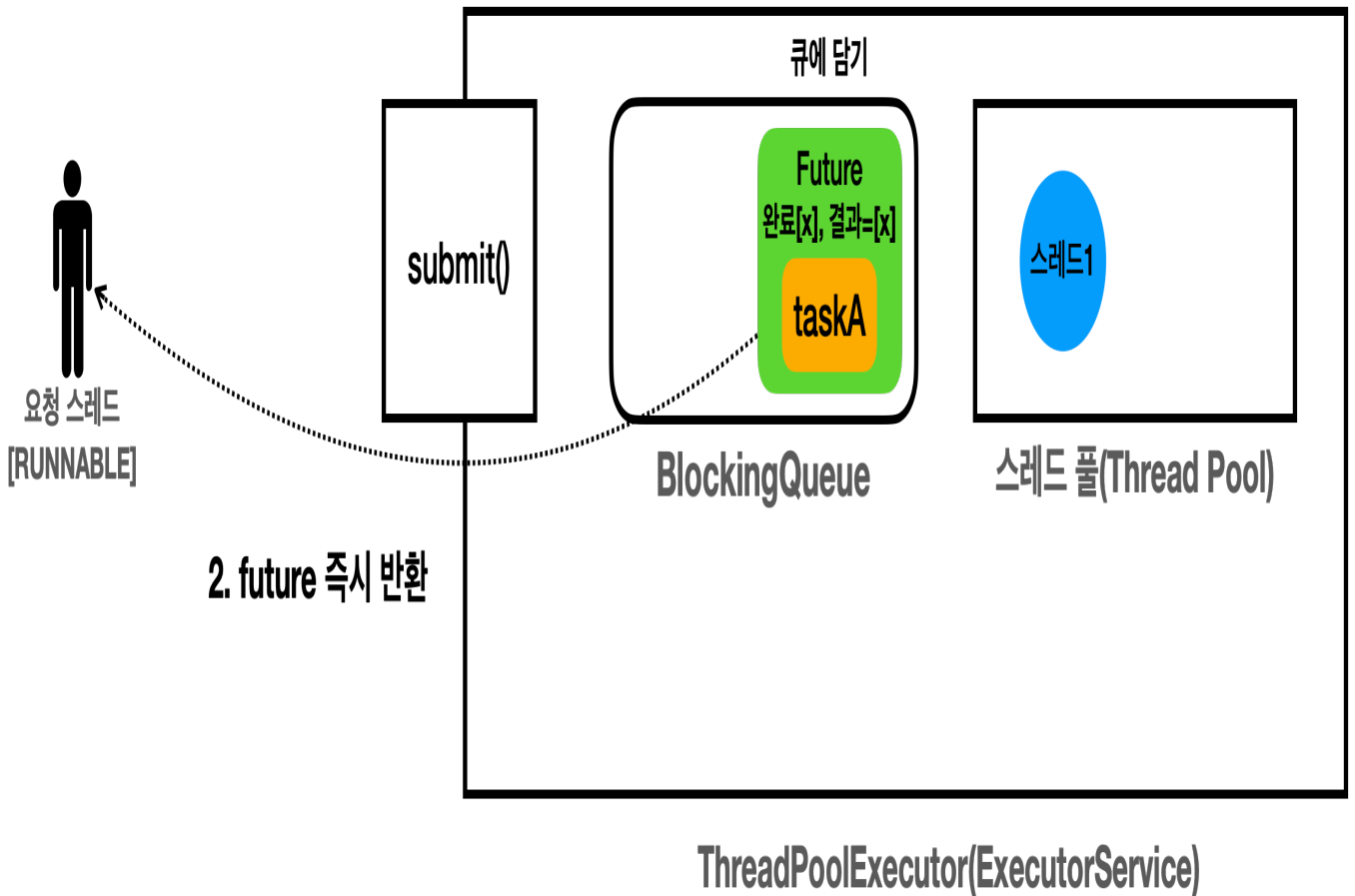
- submit() 을 호출해서 ExecutorService 에 taskA 를 전달한다.

## Future의 생성



### ThreadPoolExecutor(ExecutorService)

- 요청 스레드는 `es.submit(taskA)` 를 호출하고 있는 중이다.
- `ExecutorService` 는 전달한 `taskA` 의 미래 결과를 알 수 있는 `Future` 객체를 생성한다.
  - `Future` 는 인터페이스이다. 이때 생성되는 실제 구현체는 `FutureTask` 이다.
- 그리고 생성한 `Future` 객체 안에 `taskA`의 인스턴스를 보관한다.
- `Future` 는 내부에 `taskA` 작업의 완료 여부와, 작업의 결과 값을 가진다.



- `submit()` 을 호출한 경우 `Future` 가 만들어지고, 전달한 작업인 `taskA` 가 바로 블로킹 큐에 담기는 것이 아니라, 그림처럼 `taskA` 를 감싸고 있는 `Future` 가 대신 블로킹 큐에 담긴다.

```
Future<Integer> future = es.submit(new MyCallable());
```

```
09:24:42.691 [    main] future 즉시 반환, future = FutureTask@46d56d67[Not
completed, task = thread.executor.future.CallableMainV2$MyCallable@14acaea5]
```

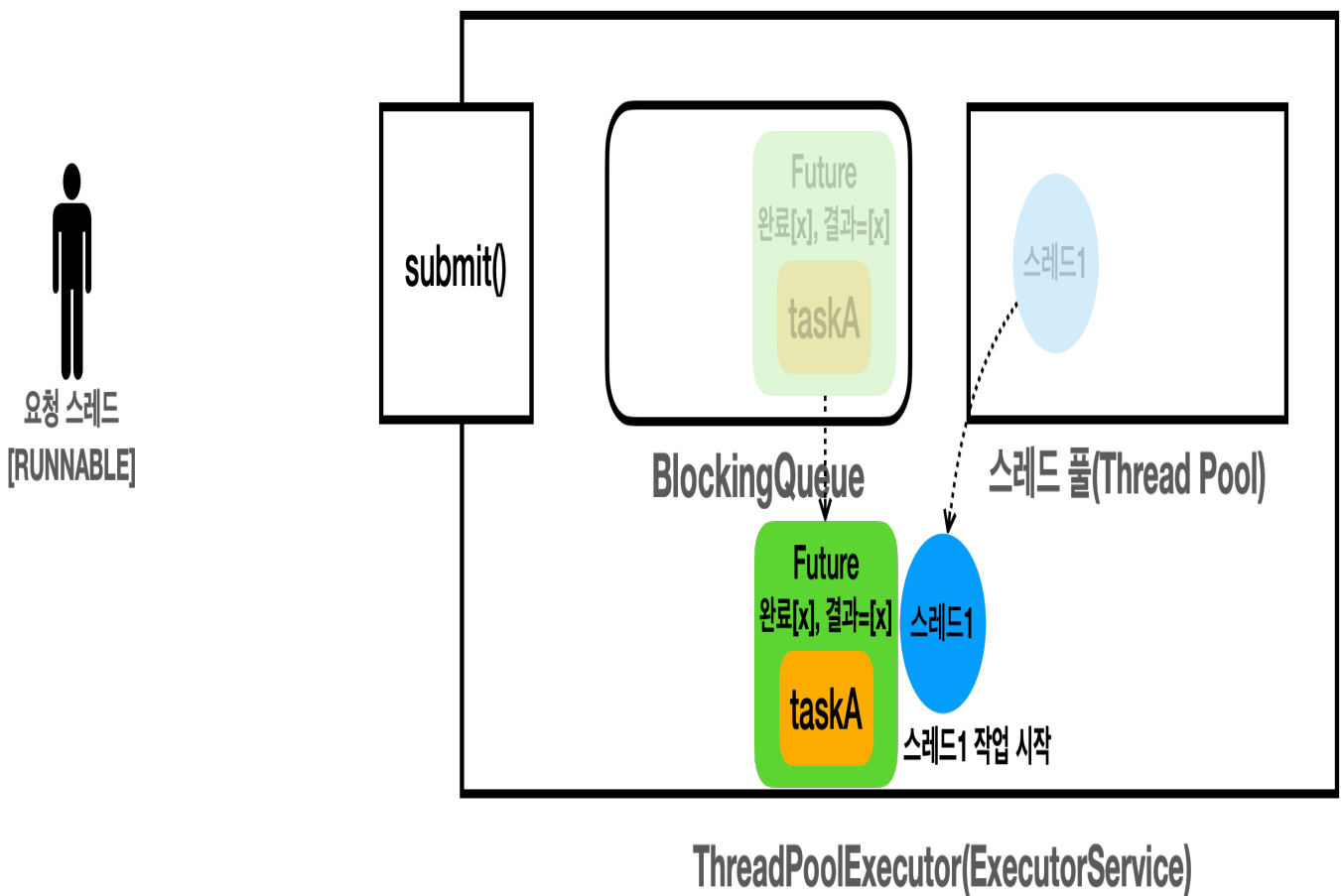
- `Future` 는 내부에 작업의 완료 여부와, 작업의 결과 값을 가진다. 작업이 완료되지 않았기 때문에 아직은 결과 값이 없다.
  - 로그를 보면 `Future` 의 구현체는 `FutureTask` 이다.
  - `Future` 의 상태는 "Not completed"(미 완료)이고, 연관된 작업은 전달한 `taskA(MyCallable 인스턴스)` 이다.
- 여기서 중요한 핵심이 있는데, 작업을 전달할 때 생성된 `Future` 는 즉시 반환된다는 점이다.

다음 로그를 보자.

```
09:24:42.691 [    main] future 즉시 반환, future = FutureTask@46d56d67[Not
```

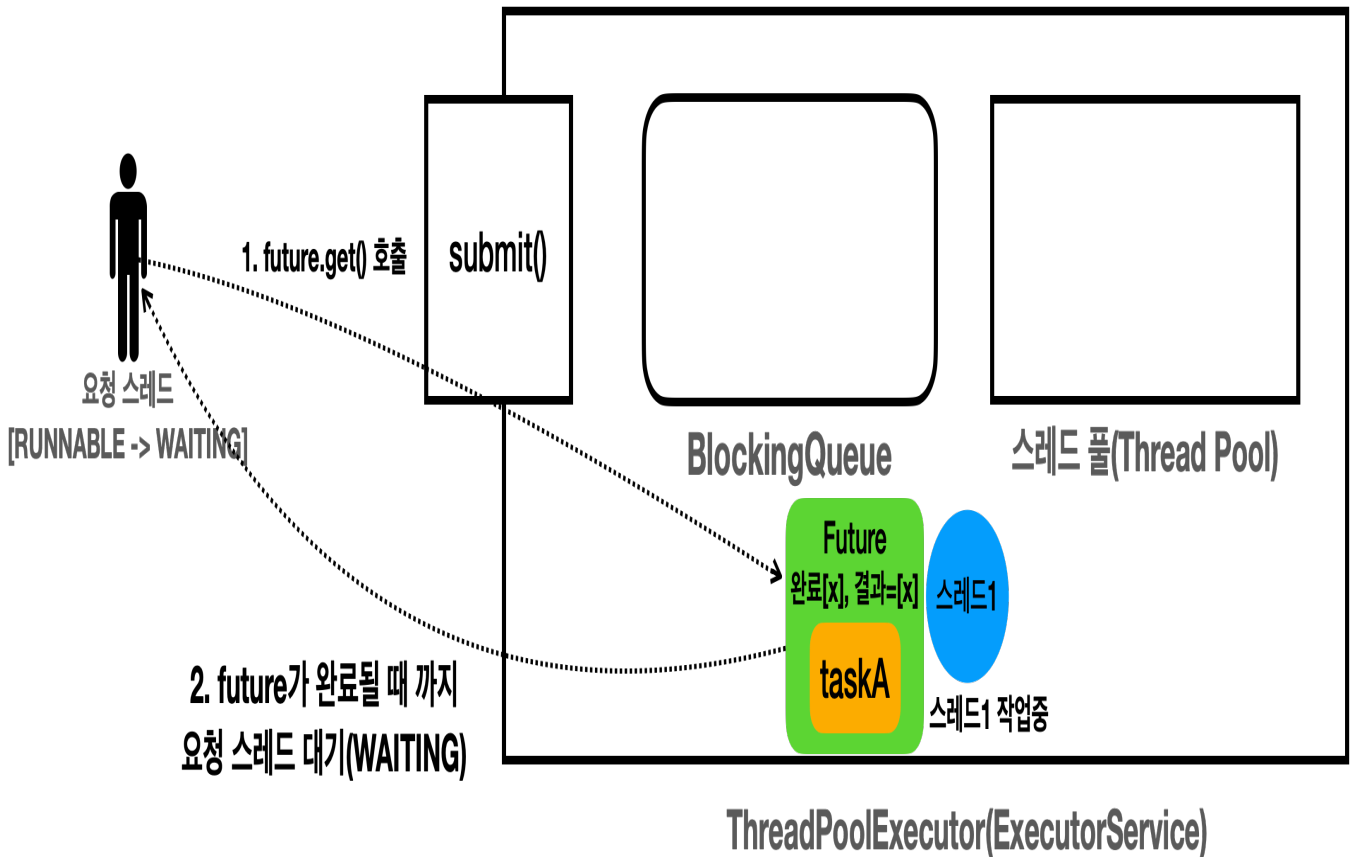
```
completed, task = thread.executor.future.CallableMainV2$MyCallable@14acaea5]
09:24:42.691 [      main] future.get() [블로킹] 메서드 호출 시작 -> main 스레드 WAITING
```

- 생성한 `Future`를 즉시 반환하기 때문에 요청 스레드는 대기하지 않고, 자유롭게 본인의 다음 코드를 호출할 수 있다.
  - 이것은 마치 `Thread.start()`를 호출한 것과 비슷하다. `Thread.start()`를 호출하면 스레드의 작업 코드가 별도의 스레드에서 실행된다. 요청 스레드는 대기하지 않고, 즉시 다음 코드를 호출할 수 있다.



```
09:24:42.691 [pool-1-thread-1] Callable 시작
```

- 큐에 들어있는 `Future[taskA]`를 꺼내서 스레드 풀의 스레드1이 작업을 시작한다.
- 참고로 `Future`의 구현체인 `FutureTask`는 `Runnable` 인터페이스도 함께 구현하고 있다.
- 스레드1은 `FutureTask`의 `run()` 메서드를 수행한다.
- 그리고 `run()` 메서드가 `taskA`의 `call()` 메서드를 호출하고 그 결과를 받아서 처리한다.
  - `FutureTask.run()` → `MyCallable.call()`



09:24:42.691 [ main] future.get() [블로킹] 메서드 호출 시작 -> main 스레드 WAITING

## 스레드1

- 스레드1은 taskA의 작업을 아직 처리중이다. 아직 완료하지는 않았다.

## 요청 스레드

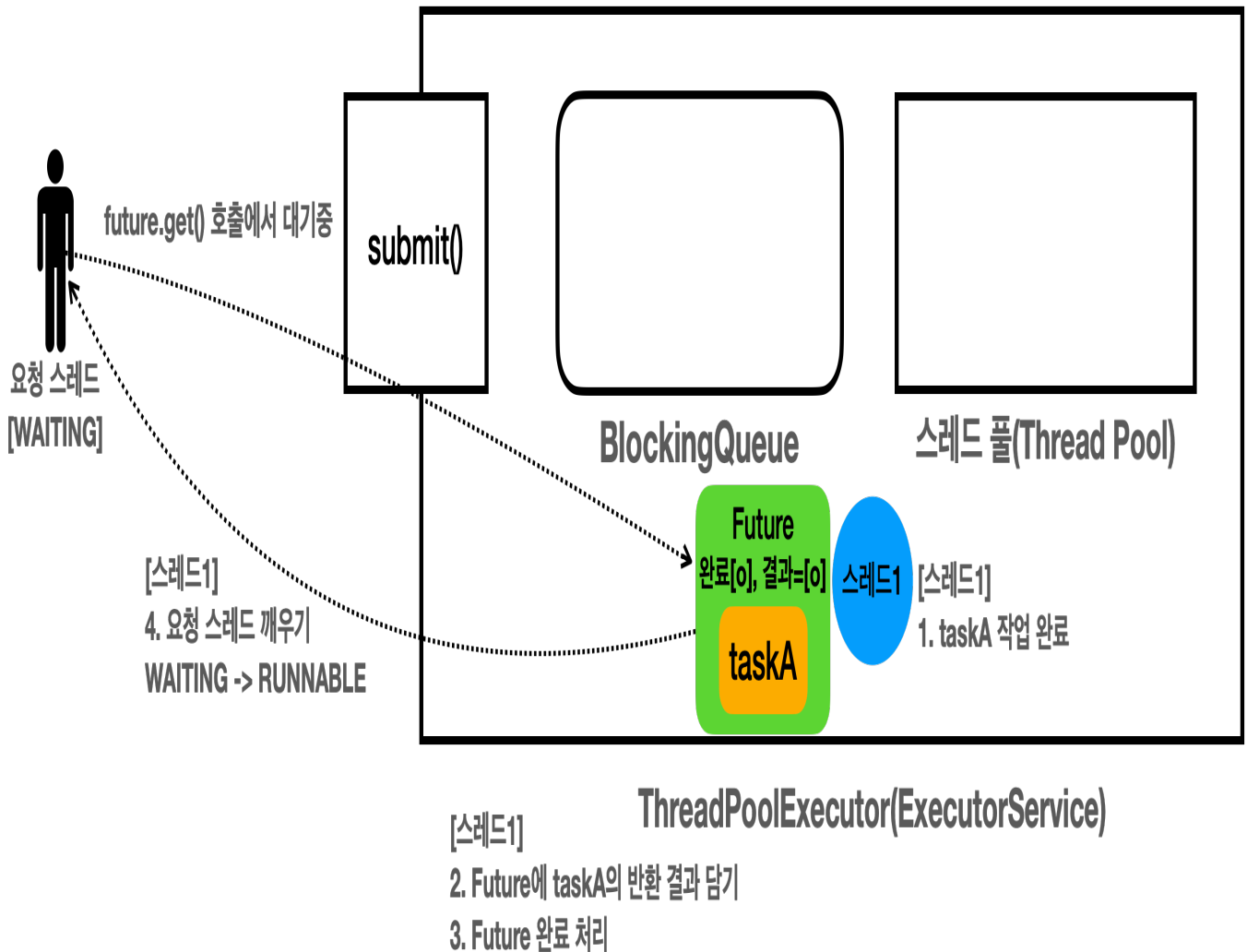
- 요청 스레드는 Future 인스턴스의 참조를 가지고 있다.
- 그리고 언제든지 본인이 필요할 때 Future.get()을 호출해서 taskA 작업의 미래 결과를 받을 수 있다.
- 요청 스레드는 작업의 결과가 필요해서 future.get()을 호출한다.
  - Future에는 완료 상태가 있다. taskA의 작업이 완료되면 Future의 상태도 완료로 변경된다.
  - 그런데 여기서 taskA의 작업이 아직 완료되지 않았다. 따라서 Future도 완료 상태가 아니다.
- 요청 스레드가 future.get()을 호출하면 Future가 완료 상태가 될 때 까지 대기한다. 이때 요청 스레드의 상태는 RUNNABLE → WAITING이 된다.

## future.get()을 호출했을 때

- Future가 완료 상태:** Future가 완료 상태면 Future에 결과도 포함되어 있다. 이 경우 요청 스레드는 대기하지 않고, 값을 즉시 반환받을 수 있다.
- Future가 완료 상태가 아님:** taskA가 아직 수행되지 않았거나 또는 수행 중이라는 뜻이다. 이때는 어쩔 수 없이 요청 스레드가 결과를 받기 위해 대기해야 한다. 요청 스레드가 마치 락을 얻을 때처럼, 결과를 얻기 위해 대기한다. 이처럼 스레드가 어떤 결과를 얻기 위해 대기하는 것을 블로킹(Blocking)이라 한다.

## 참고: 블로킹 메서드

`Thread.join()`, `Future.get()` 과 같은 메서드는 스레드가 작업을 바로 수행하지 않고, 다른 작업이 완료될 때까지 기다리게 하는 메서드이다. 이러한 메서드를 호출하면 호출한 스레드는 지정된 작업이 완료될 때까지 블록(대기)되어 다른 작업을 수행할 수 없다.



```
09:24:44.703 [pool-1-thread-1] create value = 4
09:24:44.703 [pool-1-thread-1] Callable 완료
09:24:44.703 [      main] future.get() [블로킹] 메서드 호출 완료 -> , main 스레드 RUNNABLE
```

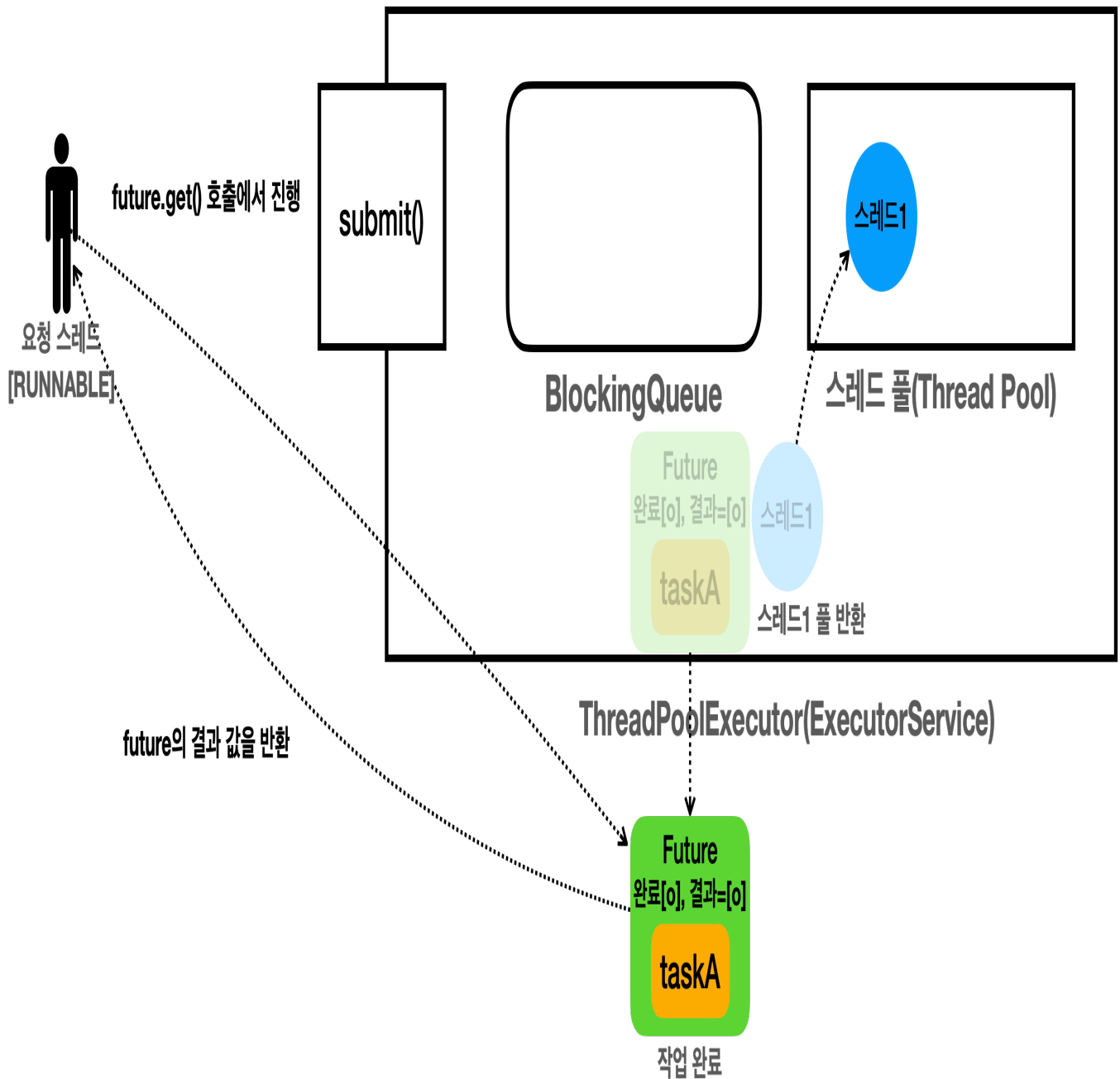
## 요청 스레드

- 대기(WAITING) 상태로 `future.get()` 을 호출하고 대기중이다.

## 스레드1

- taskA 작업을 완료한다.

2. Future 에 taskA 의 반환 결과를 담는다.
3. Future 의 상태를 완료로 변경한다.
4. 요청 스레드를 깨운다. 요청 스레드는 WAITING → RUNNABLE 상태로 변한다.



```
09:24:44.703 [    main] future.get() [블로킹] 메서드 호출 완료 -> , main 스레드
RUNNABLE
09:24:44.704 [    main] result value = 4
```

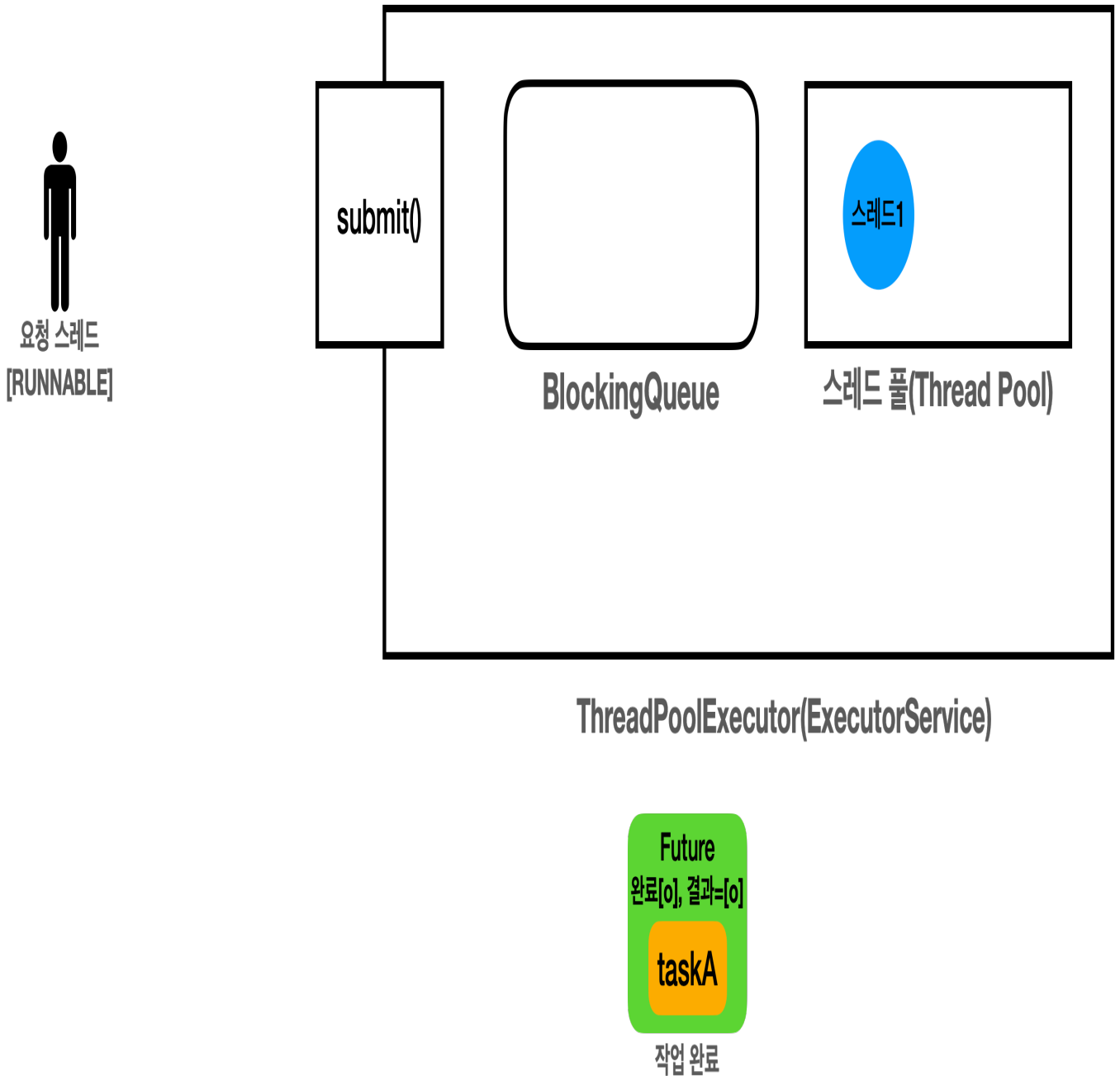
#### 요청 스레드

- 요청 스레드는 RUNNABLE 상태가 되었다. 그리고 완료 상태의 Future에서 결과를 반환 받는다. 참고로

taskA의 결과가 Future에 담겨있다.

#### 스레드1

- 작업을 마친 스레드1은 스레드 풀로 반환된다. RUNNABLE → WAITING



```
09:24:44.704 [      main] future 완료, future =  
java.util.concurrent.FutureTask@46d56d67[Completed normally]
```

- `Future`의 인스턴스인 `FutureTask`를 보면 "Completed normally"로 정상 완료된 것을 확인할 수 있다.

#### 정리

```
Future<Integer> future = es.submit(new MyCallable());
```

- `Future`는 작업의 미래 결과를 받을 수 있는 객체이다.
- `submit()` 호출시 `future`는 즉시 반환된다. 덕분에 요청 스레드는 블로킹 되지 않고, 필요한 작업을 할 수 있다.

```
Integer result = future.get();
```

- 작업의 결과가 필요하면 `Future.get()`을 호출하면 된다.
- **Future가 완료 상태:** `Future`가 완료 상태면 `Future`에 결과도 포함되어 있다. 이 경우 요청 스레드는 대기하지 않고, 값을 즉시 반환받을 수 있다.
- **Future가 완료 상태가 아님:** 작업이 아직 수행되지 않았거나 또는 수행 중이라는 뜻이다. 이때는 어쩔 수 없이 요청 스레드가 결과를 받기 위해 블로킹 상태로 대기해야 한다.

## Future가 필요한 이유?

그런데 잘 생각해보면 한 가지 의문이 들 수 있다.

다음 두 코드를 비교해보자.

### Future를 반환 하는 코드

```
Future<Integer> future = es.submit(new MyCallable()); // 여기는 블로킹 아님
future.get(); // 여기서 블로킹
```

`ExecutorService`를 설계할 때 지금까지처럼 복잡하게 `Future`를 반환하는게 아니라 다음과 같이 결과를 직접 받도록 설계하는게 더 단순하고 좋지 않았을까?

### 결과를 직접 반환 하는 코드(가정)

```
Integer result = es.submit(new MyCallable()); // 여기서 블로킹
```

물론 이렇게 설계하면 `submit()`을 호출할 때, 작업의 결과가 언제 나올지 알 수 없다. 따라서 작업의 결과를 받을 때까지 요청 스레드는 대기해야 한다. 그런데 이것은 `Future`를 사용할 때도 마찬가지다. `Future`만 즉시 반환 받을 뿐이지, 작업의 결과를 얻으려면 결국 `future.get()`을 호출해야 한다. 그리고 이 시점에는 작업의 결과를 받을 때 까

지 대기해야 한다.

다음 활용 예제를 보면 `Future` 라는 개념이 왜 필요한지 이해가 될 것이다.

## Future3 - 활용

이번에는 숫자를 나누어 더하는 기능을 멀티스레드로 수행해보자.

1~100 까지 더하는 경우를 스레드를 사용해서 1~50, 51~100 으로 나누어 처리해보자.

- 스레드1: 1~50 까지 더함
- 스레드2: 51~100 까지 더함

## SumTask - Runnable

먼저 `ExecutorService` 없이 `Runnable` 과 순수 스레드로 수행해보자.

```
package thread.executor.future;

import static util.MyLogger.log;

public class SumTaskMainV1 {

    public static void main(String[] args) throws InterruptedException {
        SumTask task1 = new SumTask(1, 50);
        SumTask task2 = new SumTask(51, 100);
        Thread thread1 = new Thread(task1, "thread-1");
        Thread thread2 = new Thread(task2, "thread-2");

        thread1.start();
        thread2.start();

        //스레드가 종료될 때 까지 대기
        log("join() - main 스레드가 thread1, thread2 종료까지 대기");
        thread1.join();
        thread2.join();
        log("main 스레드 대기 완료");

        log("task1.result=" + task1.result);
        log("task2.result=" + task2.result);
    }
}
```

```

        int sumAll = task1.result + task2.result;
        log("task1 + task2 = " + sumAll);
        log("End");
    }

    static class SumTask implements Runnable {
        int startValue;
        int endValue;
        int result = 0;

        public SumTask(int startValue, int endValue) {
            this.startValue = startValue;
            this.endValue = endValue;
        }

        @Override
        public void run() {
            log("작업 시작");

            try {
                Thread.sleep(2000);
            } catch (InterruptedException e) {
                throw new RuntimeException(e);
            }

            int sum = 0;
            for (int i = startValue; i <= endValue; i++) {
                sum += i;
            }
            result = sum;

            log("작업 완료 result=" + result);
        }
    }
}

```

## 실행 결과

```

11:05:46.862 [ thread-2] 작업 시작
11:05:46.862 [ thread-1] 작업 시작

```

```
11:05:46.862 [      main] join() - main 스레드가 thread1, thread2 종료까지 대기
11:05:48.872 [ thread-2] 작업 완료 result=3775
11:05:48.872 [ thread-1] 작업 완료 result=1275
11:05:48.873 [      main] main 스레드 대기 완료
11:05:48.873 [      main] task1.result=1275
11:05:48.873 [      main] task2.result=3775
11:05:48.873 [      main] task1 + task2 = 5050
11:05:48.873 [      main] End
```

이미 앞서 처리해보았던 문제여서 크게 어려움은 없을 것이다.

이제 이 코드를 `Callable` 과 `ExecutorService` 로 처리해보자.

## SumTask - Callable

이번에는 앞의 코드를 `ExecutorService` 와 `Callable` 로 처리해보자.

```
package thread.executor.future;

import java.util.concurrent.*;

import static util.MyLogger.log;

public class SumTaskMainV2 {

    public static void main(String[] args) throws InterruptedException,
    ExecutionException {
        SumTask task1 = new SumTask(1, 50);
        SumTask task2 = new SumTask(51, 100);
        ExecutorService es = Executors.newFixedThreadPool(2);

        Future<Integer> future1 = es.submit(task1);
        Future<Integer> future2 = es.submit(task2);

        Integer sum1 = future1.get();
        Integer sum2 = future2.get();

        log("task1.result=" + sum1);
        log("task2.result=" + sum2);

        int sumAll = sum1 + sum2;
```

```

        log("task1 + task2 = " + sumAll);
        log("End");

        es.close();
    }

    static class SumTask implements Callable<Integer> {
        int startValue;
        int endValue;

        public SumTask(int startValue, int endValue) {
            this.startValue = startValue;
            this.endValue = endValue;
        }

        @Override
        public Integer call() throws InterruptedException {
            log("작업 시작");
            Thread.sleep(2000);
            int sum = 0;
            for (int i = startValue; i <= endValue; i++) {
                sum += i;
            }
            log("작업 완료 result=" + sum);
            return sum;
        }
    }
}

```

## 실행 결과

```

11:10:08.134 [pool-1-thread-1] 작업 시작
11:10:08.134 [pool-1-thread-2] 작업 시작
11:10:10.149 [pool-1-thread-1] 작업 완료 result=1275
11:10:10.149 [pool-1-thread-2] 작업 완료 result=3775
11:10:10.150 [      main] task1.result=1275
11:10:10.153 [      main] task2.result=3775
11:10:10.153 [      main] task1 + task2 = 5050
11:10:10.154 [      main] End

```

`ExecutorService`와 `Callable`을 사용한 덕분에, 이전 코드보다 훨씬 직관적이고 깔끔하게 코드를 작성할 수 있었다.

특히 작업의 결과를 반환하고, 요청 스레드에서 그 결과를 바로 받아서 처리하는 부분이 매우 직관적이다. 코드만 보면 마치 멀티스레드를 사용하지 않고, 단일 스레드 상황에서 일반적인 메서드를 호출하고 결과를 받는 것 처럼 느껴진다. 그리고 스레드를 생성하고, `Thread.join()`과 같은 스레드를 관리하는 코드도 모두 제거할 수 있었다. 추가로 `Callable.call()`은 `throws InterruptedException`과 같은 체크 예외도 던질 수 있다.

## Future4 - 이유

### Future가 필요한 이유

이제 `Future`가 필요한 이유를 이번 코드를 통해 알아보자.

다음 두 코드를 비교해보자.

#### Future를 반환 하는 코드

```
Future<Integer> future1 = es.submit(task1); // 여기는 블로킹 아님
Future<Integer> future2 = es.submit(task2); // 여기는 블로킹 아님

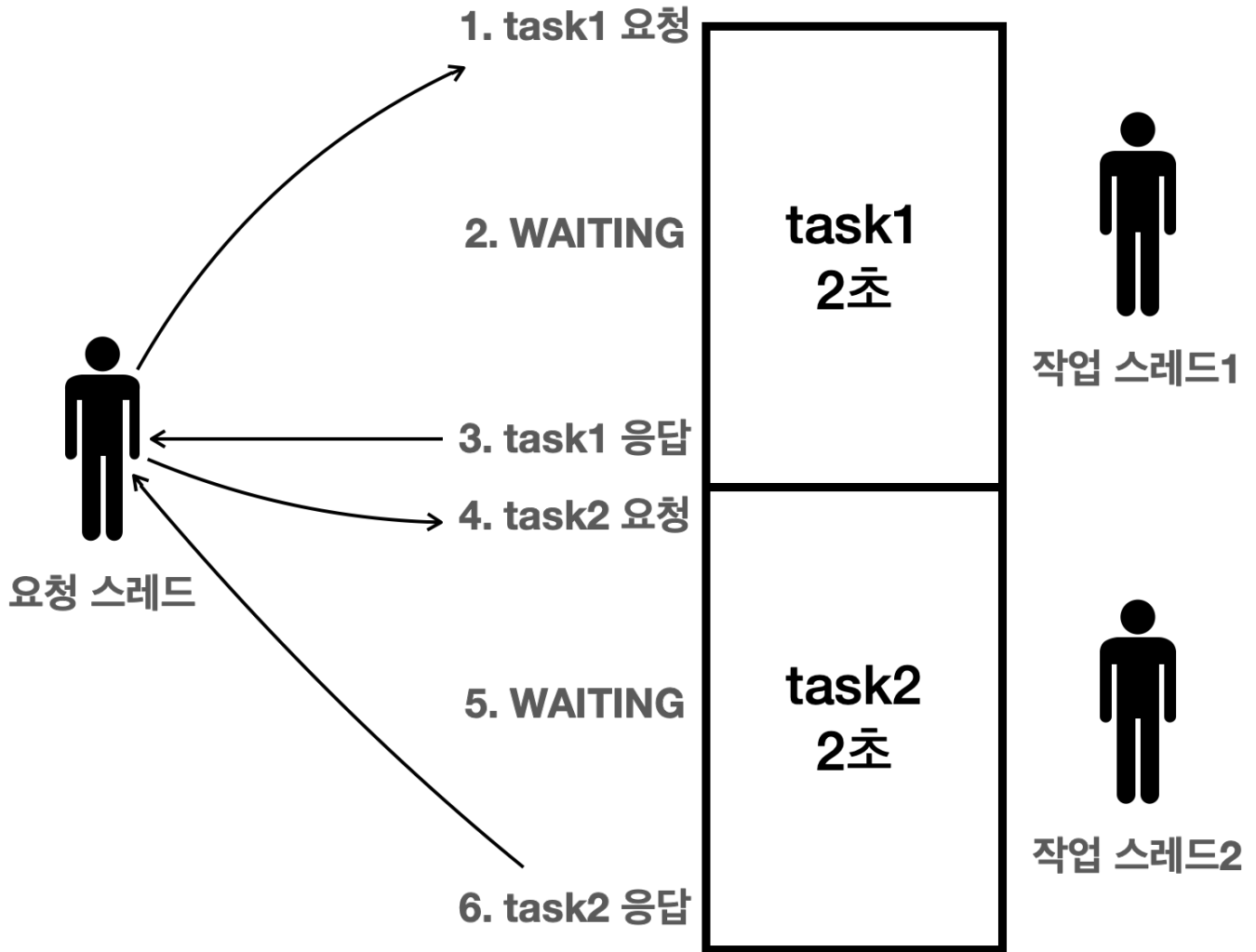
Integer sum1 = future1.get(); // 여기서 블로킹
Integer sum2 = future2.get(); // 여기서 블로킹
```

#### Future 없이 결과를 직접 반환 하는 코드(가정)

```
Integer sum1 = es.submit(task1); // 여기서 블로킹
Integer sum2 = es.submit(task2); // 여기서 블로킹
```

- 참고로 이 코드는 가정이다. 이런 코드는 없다.

#### Future 없이 결과를 직접 반환 - 가정

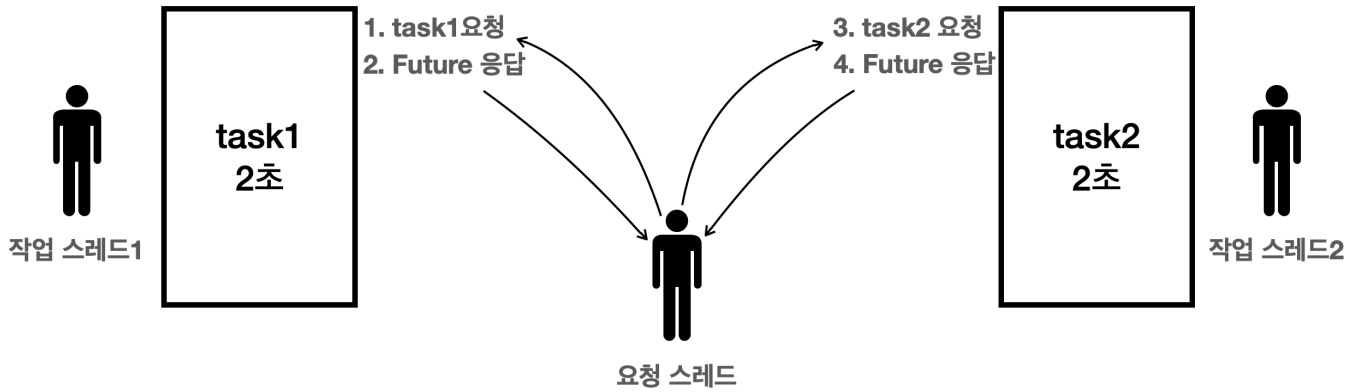


먼저 `ExecutorService`가 `Future` 없이 결과를 직접 반환한다고 가정해보자.

- 요청 스레드는 `task1`을 `ExecutorService`에 요청하고 결과를 기다린다.
  - 작업 스레드가 작업을 수행하는데 2초가 걸린다.
  - 요청 스레드는 결과를 받을 때 까지 2초간 대기한다.
  - 요청 스레드는 2초 후에 결과를 받고 다음 라인을 수행한다.
- 요청 스레드는 `task2`을 `ExecutorService`에 요청하고 결과를 기다린다.
  - 작업 스레드가 작업을 수행하는데 2초가 걸린다.
  - 요청 스레드는 결과를 받을 때 까지 2초간 대기한다.
  - 결과를 받고 요청 스레드가 다음 라인을 수행한다.

`Future`를 사용하지 않는 경우 결과적으로 `task1`의 결과를 기다린 다음에 `task2`를 요청한다. 따라서 총 4초의 시간이 걸렸다. 이것은 마치 단일 스레드가 작업을 한 것과 비슷한 결과이다!

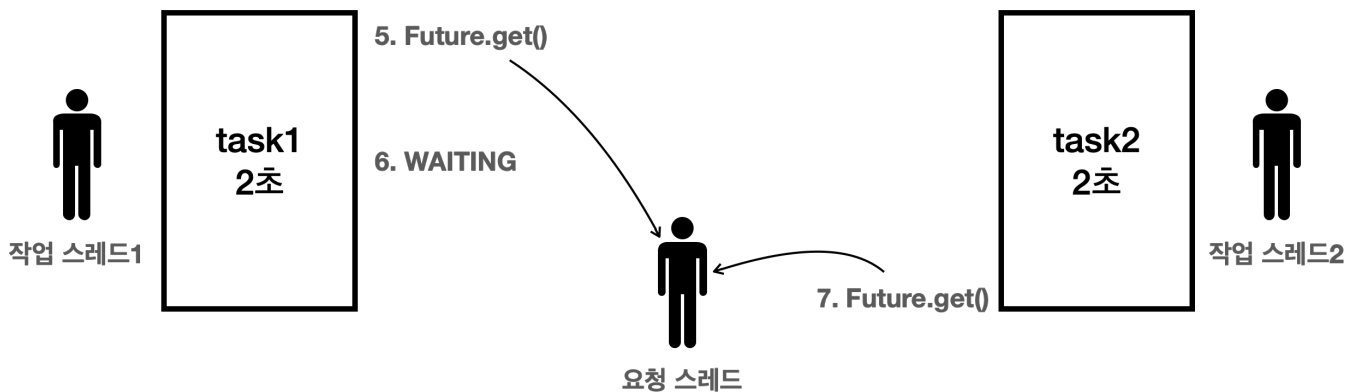
**Future를 반환**



이번에는 Future를 반환한다고 가정해보자.

- 요청 스레드는 task1을 ExecutorService에 요청한다.
  - 요청 스레드는 즉시 Future를 반환 받는다.
  - 작업 스레드1은 task1을 수행한다.
- 요청 스레드는 task2을 ExecutorService에 요청한다.
  - 요청 스레드는 즉시 Future를 반환 받는다.
  - 작업 스레드2는 task2을 수행한다.

요청 스레드는 task1, task2를 동시에 요청할 수 있다. 따라서 두 작업은 동시에 수행된다.



- 이후에 요청 스레드는 future1.get()을 호출하며 대기한다.
  - 작업 스레드1이 작업을 진행하는 약 2초간 대기하고 결과를 받는다.
- 이후에 요청 스레드는 future2.get()을 호출하며 즉시 결과를 받는다.
  - 작업 스레드2는 이미 2초간 작업을 완료했다. 따라서 future2.get()은 거의 즉시 결과를 반환한다.

## Future를 잘못 사용하는 예

앞서 설명한 문제 상황과 같은 원리로 Future를 호출하자마자 바로 get()을 호출해도 문제가 될 수 있다.

## Future를 적절하게 잘 활용

```
Future<Integer> future1 = es.submit(task1); // non-blocking
Future<Integer> future2 = es.submit(task2); // non-blocking
```

```
Integer sum1 = future1.get(); // blocking, 2초 대기
Integer sum2 = future2.get(); // blocking, 즉시 반환
```

- 요청 스레드가 필요한 작업을 모두 요청한 다음에 결과를 받는다.
- 총 2초의 시간이 걸린다.

### Future를 잘못 활용한 예1

```
Future<Integer> future1 = es.submit(task1); // non-blocking
Integer sum1 = future1.get(); // blocking, 2초 대기

Future<Integer> future2 = es.submit(task2); // non-blocking
Integer sum2 = future2.get(); // blocking, 2초 대기
```

- 요청 스레드가 작업을 하나 요청하고 그 결과를 기다린다. 그리고 그 다음에 다시 다음 요청을 전달하고 결과를 기다린다.
- 총 4초의 시간이 걸린다.

### Future를 잘못 활용한 예2

```
Integer sum1 = es.submit(task1).get(); // get()에서 블로킹
Integer sum2 = es.submit(task2).get(); // get()에서 블로킹
```

- Future를 잘못 활용한 예1과 똑같은 코드이다. 대신에 `submit()`을 호출하고 그 결과를 변수에 담지 않고 바로 연결해서 `get()`을 호출한다.
- 총 4초의 시간이 걸린다.

실제 4초의 시간이 걸리는지 코드로 확인해보자.

`SumTaskMainV2`를 복사해서 `SumTaskMainV2_Bad`를 만들자.

```
package thread.executor.future;

import java.util.concurrent.*;

import static util.MyLogger.log;

public class SumTaskMainV2_Bad {
```

```

    public static void main(String[] args) throws InterruptedException,
    ExecutionException {
        SumTask task1 = new SumTask(1, 50);
        SumTask task2 = new SumTask(51, 100);
        ExecutorService es = Executors.newFixedThreadPool(2);

        Future<Integer> future1 = es.submit(task1); // non-blocking
        Integer sum1 = future1.get(); // blocking, 2초 대기
        Future<Integer> future2 = es.submit(task2); // non-blocking
        Integer sum2 = future2.get(); // blocking, 2초 대기

        log("task1.result=" + sum1);
        log("task2.result=" + sum2);

        int sumAll = sum1 + sum2;
        log("task1 + task2 = " + sumAll);
        log("End");

        es.close();
    }

    static class SumTask implements Callable<Integer> {
        int startValue;
        int endValue;

        public SumTask(int startValue, int endValue) {
            this.startValue = startValue;
            this.endValue = endValue;
        }

        @Override
        public Integer call() throws InterruptedException {
            log("작업 시작");
            Thread.sleep(2000);
            int sum = 0;
            for (int i = startValue; i <= endValue; i++) {
                sum += i;
            }
            log("작업 완료 result=" + sum);
            return sum;
        }
    }
}

```

```
}
```

## 실행 결과

```
13:56:16.439 [pool-1-thread-1] 작업 시작
13:56:18.456 [pool-1-thread-1] 작업 완료 result=1275
13:56:18.458 [pool-1-thread-2] 작업 시작
13:56:20.464 [pool-1-thread-2] 작업 완료 result=3775
13:56:20.465 [      main] task1.result=1275
13:56:20.466 [      main] task2.result=3775
13:56:20.466 [      main] task1 + task2 = 5050
13:56:20.466 [      main] End
```

총 4초의 시간이 걸린 것을 확인할 수 있다.

## 정리

- `Future` 라는 개념이 없다면 결과를 받을 때 까지 요청 스레드는 아무일도 못하고 대기해야 한다. 따라서 다른 작업을 동시에 수행할 수도 없다.
- `Future` 라는 개념 덕분에 요청 스레드는 대기하지 않고, 다른 작업을 수행할 수 있다. 예를 들어서 다른 작업을 더 요청할 수 있다. 그리고 모든 작업 요청이 끝난 다음에, 본인이 필요할 때 `Future.get()` 을 호출해서 최종 결과를 받을 수 있다.
- `Future` 를 사용하는 경우 결과적으로 `task1`, `task2` 를 동시에 요청할 수 있다. 두 작업을 바로 요청했기 때문에 작업을 동시에 제대로 수행할 수 있다.

`Future` 는 요청 스레드를 블로킹(대기) 상태로 만들지 않고, 필요한 요청을 모두 수행할 수 있게 해준다. 필요한 모든 요청을 한 다음에 `Future.get()` 을 통해 블로킹 상태로 대기하며 결과를 받으면 된다.

이런 이유로 `ExecutorService` 는 결과를 직접 반환하지 않고, `Future` 를 반환한다.

## Future5 - 정리

`Future` 는 작업의 미래 계산의 결과를 나타내며, 계산이 완료되었는지 확인하고, 완료될 때까지 기다릴 수 있는 기능을 제공한다.

## Future 인터페이스

```
package java.util.concurrent;

public interface Future<V> {

    boolean cancel(boolean mayInterruptIfRunning);
    boolean isCancelled();

    boolean isDone();

    V get() throws InterruptedException, ExecutionException;
    V get(long timeout, TimeUnit unit)
        throws InterruptedException, ExecutionException, TimeoutException;

    enum State {
        RUNNING,
        SUCCESS,
        FAILED,
        CANCELLED
    }

    default State state() {}
}
```

## 주요 메서드

### **boolean cancel(boolean mayInterruptIfRunning)**

- **기능:** 아직 완료되지 않은 작업을 취소한다.
- **매개변수:** mayInterruptIfRunning
  - **cancel(true):** Future를 취소 상태로 변경한다. 이때 작업이 실행중이라면 Thread.interrupt()를 호출해서 작업을 중단한다.
  - **cancel(false):** Future를 취소 상태로 변경한다. 단 이미 실행 중인 작업을 중단하지는 않는다.
- **반환값:** 작업이 성공적으로 취소된 경우 true, 이미 완료되었거나 취소할 수 없는 경우 false
- **설명:** 작업이 실행 중이 아니거나 아직 시작되지 않았으면 취소하고, 실행 중인 작업의 경우 mayInterruptIfRunning이 true이면 중단을 시도한다.
- **참고:** 취소 상태의 Future에 Future.get()을 호출하면 CancellationException 런타임 예외가 발생한다.

### **boolean isCancelled()**

- **기능:** 작업이 취소되었는지 여부를 확인한다.
- **반환값:** 작업이 취소된 경우 `true`, 그렇지 않은 경우 `false`
- **설명:** 이 메서드는 작업이 `cancel()` 메서드에 의해 취소된 경우에 `true`를 반환한다.

### **boolean isDone()**

- **기능:** 작업이 완료되었는지 여부를 확인한다.
- **반환값:** 작업이 완료된 경우 `true`, 그렇지 않은 경우 `false`
- **설명:** 작업이 정상적으로 완료되었거나, 취소되었거나, 예외가 발생하여 종료된 경우에 `true`를 반환한다.

### **State state()**

- **기능:** Future의 상태를 반환한다. 자바 19부터 지원한다.
  - `RUNNING`: 작업 실행 중
  - `SUCCESS`: 성공 완료
  - `FAILED`: 실패 완료
  - `CANCELLED`: 취소 완료

### **V get()**

- **기능:** 작업이 완료될 때까지 대기하고, 완료되면 결과를 반환한다.
- **반환값:** 작업의 결과
- **예외**
  - `InterruptedException`: 대기 중에 현재 스레드가 인터럽트된 경우 발생
  - `ExecutionException`: 작업 계산 중에 예외가 발생한 경우 발생
- **설명:** 작업이 완료될 때까지 `get()`을 호출한 현재 스레드를 대기(블록킹)한다. 작업이 완료되면 결과를 반환한다.

### **V get(long timeout, TimeUnit unit)**

- **기능:** `get()`과 같은데, 시간 초과되면 예외를 발생시킨다.
- **매개변수:**
  - `timeout`: 대기할 최대 시간
  - `unit`: `timeout` 매개변수의 시간 단위 지정
- **반환값:** 작업의 결과
- **예외:**
  - `InterruptedException`: 대기 중에 현재 스레드가 인터럽트된 경우 발생
  - `ExecutionException`: 계산 중에 예외가 발생한 경우 발생
  - `TimeoutException`: 주어진 시간 내에 작업이 완료되지 않은 경우 발생
- **설명:** 지정된 시간 동안 결과를 기다린다. 시간이 초과되면 `TimeoutException`을 발생시킨다.

## Future6 - 취소

cancel() 이 어떻게 동작하는지 알아보자.

```
package thread.executor.future;

import java.util.concurrent.*;

import static util.MyLogger.log;
import static util.ThreadUtils.sleep;

public class FutureCancelMain {

    private static boolean mayInterruptIfRunning = true; // 변경
    //private static boolean mayInterruptIfRunning = false; // 변경

    public static void main(String[] args) {
        ExecutorService es = Executors.newFixedThreadPool(1);
        Future<String> future = es.submit(new MyTask());
        log("Future.state: " + future.state());

        // 일정 시간 후 취소 시도
        sleep(3000);

        // cancel() 호출
        log("future.cancel(" + mayInterruptIfRunning + ") 호출");
        boolean cancelResult1 = future.cancel(mayInterruptIfRunning);
        log("Future.state: " + future.state());
        log("cancel(" + mayInterruptIfRunning + ") result: " + cancelResult1);

        // 결과 확인
        try {
            log("Future result: " + future.get());
        } catch (CancellationException e) { // 런타임 예외
            log("Future는 이미 취소 되었습니다.");
        } catch (InterruptedException | ExecutionException e) {
            e.printStackTrace();
        }
    }
}
```

```

        // Executor 종료
        es.close();
    }

    static class MyTask implements Callable<String> {

        @Override
        public String call() {
            try {
                for (int i = 0; i < 10; i++) {
                    log("작업 중: " + i);
                    Thread.sleep(1000); // 1초 동안 sleep
                }
            } catch (InterruptedException e) {
                log("인터럽트 발생");
                return "Interrupted";
            }
            return "Completed";
        }
    }
}

```

매개변수 `mayInterruptIfRunning` 를 변경하면서 어떻게 작동하는지 차이를 확인해보자.

- `cancel(true)`: Future 를 취소 상태로 변경한다. 이때 작업이 실행중이라면 `Thread.interrupt()` 를 호출해서 작업을 중단한다.
- `cancel(false)`: Future 를 취소 상태로 변경한다. 단 이미 실행 중인 작업을 중단하지는 않는다.

#### 실행 결과 - `mayInterruptIfRunning=true`

```

14:48:26.575 [      main] Future.state: RUNNING
14:48:26.575 [pool-1-thread-1] 작업 중: 0
14:48:27.583 [pool-1-thread-1] 작업 중: 1
14:48:28.589 [pool-1-thread-1] 작업 중: 2
14:48:29.580 [      main] future.cancel(true) 호출
14:48:29.581 [pool-1-thread-1] 인터럽트 발생
14:48:29.581 [      main] Future.state: CANCELLED
14:48:29.582 [      main] cancel(true) result: true
14:48:29.582 [      main] Future는 이미 취소 되었습니다.

```

- `cancel(true)` 를 호출했다.
- `mayInterruptIfRunning=true` 를 사용하면 실행중인 작업에 인터럽트가 발생해서 실행중인 작업을 중지 시도한다.
- 이후 `Future.get()` 을 호출하면 `CancellationException` 런타임 예외가 발생한다.

### 실행 결과 - `mayInterruptIfRunning=false`

```
14:48:48.257 [      main] Future.state: RUNNING
14:48:48.257 [pool-1-thread-1] 작업 중: 0
14:48:49.264 [pool-1-thread-1] 작업 중: 1
14:48:50.268 [pool-1-thread-1] 작업 중: 2
14:48:51.265 [      main] future.cancel(false) 호출
14:48:51.266 [      main] Future.state: CANCELLED
14:48:51.266 [      main] cancel(false) result: true
14:48:51.266 [      main] Future는 이미 취소 되었습니다.
14:48:51.273 [pool-1-thread-1] 작업 중: 3
14:48:52.279 [pool-1-thread-1] 작업 중: 4
14:48:53.284 [pool-1-thread-1] 작업 중: 5
14:48:54.290 [pool-1-thread-1] 작업 중: 6
14:48:55.292 [pool-1-thread-1] 작업 중: 7
14:48:56.298 [pool-1-thread-1] 작업 중: 8
14:48:57.301 [pool-1-thread-1] 작업 중: 9
```

- `cancel(false)` 를 호출했다.
- `mayInterruptIfRunning=false` 를 사용하면 실행중인 작업은 그냥 둔다. (인터럽트를 걸지 않는다.)
- 실행중인 작업은 그냥 두더라도 `cancel()` 을 호출했기 때문에 `Future` 는 `CANCEL` 상태가 된다.
- 이후 `Future.get()` 을 호출하면 `CancellationException` 런타임 예외가 발생한다.

## Future7 - 예외

`Future.get()` 을 호출하면 작업의 결과값 뿐만 아니라, 작업 중에 발생한 예외도 받을 수 있다.

```
package thread.executor.future;

import java.util.concurrent.*;
```

```

import static util.MyLogger.log;
import static util.ThreadUtils.sleep;

public class FutureExceptionMain {

    public static void main(String[] args) {
        ExecutorService es = Executors.newFixedThreadPool(1);
        log("작업 전달");
        Future<Integer> future = es.submit(new ExCallable());
        sleep(1000); // 잠시 대기

        try {
            log("future.get() 호출 시도, future.state(): " + future.state());
            Integer result = future.get();
            log("result value = " + result);
        } catch (InterruptedException e) {
            throw new RuntimeException(e);
        } catch (ExecutionException e) {
            log("e = " + e);
            Throwable cause = e.getCause(); // 원본 예외
            log("cause = " + cause);
        }
        es.close();
    }

    static class ExCallable implements Callable<Integer> {
        @Override
        public Integer call() {
            log("Callable 실행, 예외 발생");
            throw new IllegalStateException("ex!");
        }
    }
}

```

## 실행 결과

```

15:05:04.460 [      main] 작업 전달
15:05:04.463 [pool-1-thread-1] Callable 실행, 예외 발생
15:05:05.471 [      main] future.get() 호출 시도, future.state(): FAILED
15:05:05.472 [      main] e = java.util.concurrent.ExecutionException:
java.lang.IllegalStateException: ex!
15:05:05.473 [      main] cause = java.lang.IllegalStateException: ex!

```

- **요청 스레드:** `es.submit(new ExCallable())` 을 호출해서 작업을 전달한다.
- **작업 스레드:** `ExCallable` 을 실행하는데, `IllegalStateException` 예외가 발생한다.
  - 작업 스레드는 `Future` 에 발생한 예외를 담아둔다. 참고로 예외도 객체이다. 잡아서 필드에 보관할 수 있다.
  - 예외가 발생했으므로 `Future` 의 상태는 `FAILED` 가 된다.
- **요청 스레드:** 결과를 얻기 위해 `future.get()` 을 호출한다.
  - `Future` 의 상태가 `FAILED` 면 `ExecutionException` 예외를 던진다.
  - 이 예외는 내부에 앞서 `Future` 에 저장해둔 `IllegalStateException` 을 포함하고 있다.
  - `e.getCause()` 을 호출하면 작업에서 발생한 원본 예외를 받을 수 있다.

`Future.get()` 은 작업의 결과 값을 받을 수도 있고, 예외를 받을 수도 있다. 마치 싱글 스레드 상황에서 일반적인 메서드를 호출하는 것 같다. `Executor` 프레임워크가 얼마나 잘 설계되어 있는지 알 수 있는 부분이다.

## ExecutorService - 작업 컬렉션 처리

`ExecutorService` 는 여러 작업을 한 번에 편리하게 처리하는 `invokeAll()`, `invokeAny()` 기능을 제공한다.

### 작업 컬렉션 처리

#### invokeAll()

- `<T> List<Future<T>> invokeAll(Collection<? extends Callable<T>> tasks) throws InterruptedException`
  - 모든 `Callable` 작업을 제출하고, 모든 작업이 완료될 때까지 기다린다.
- `<T> List<Future<T>> invokeAll(Collection<? extends Callable<T>> tasks, long timeout, TimeUnit unit) throws InterruptedException`
  - 지정된 시간 내에 모든 `Callable` 작업을 제출하고 완료될 때까지 기다린다.

#### invokeAny()

- `<T> T invokeAny(Collection<? extends Callable<T>> tasks) throws InterruptedException, ExecutionException`
  - 하나의 `Callable` 작업이 완료될 때까지 기다리고, 가장 먼저 완료된 작업의 결과를 반환한다.
  - 완료되지 않은 나머지 작업은 취소한다.
- `<T> T invokeAny(Collection<? extends Callable<T>> tasks, long timeout,`

TimeUnit unit) throws InterruptedException, ExecutionException, TimeoutException

- 지정된 시간 내에 하나의 Callable 작업이 완료될 때까지 기다리고, 가장 먼저 완료된 작업의 결과를 반환한다.
- 완료되지 않은 나머지 작업은 취소한다.

invokeAll(), invokeAny() 를 사용하면 한꺼번에 여러 작업을 요청할 수 있다. 둘의 차이를 코드로 알아보자.

예제를 시작하기 전에 특정 시간 대기하는 CallableTask 를 만들자. 앞서 만든 RunnableTask 의 Callable 버전이다.

```
package thread.executor;

import java.util.concurrent.Callable;

import static util.MyLogger.log;
import static util.ThreadUtils.sleep;

public class CallableTask implements Callable<Integer> {

    private String name;
    private int sleepMs = 1000;

    public CallableTask(String name) {
        this.name = name;
    }

    public CallableTask(String name, int sleepMs) {
        this.name = name;
        this.sleepMs = sleepMs;
    }

    @Override
    public Integer call() throws Exception {
        log(name + " 실행");
        sleep(sleepMs);
        log(name + " 완료, return = " + sleepMs);
        return sleepMs;
    }
}
```

- Callable 인터페이스를 구현한다.
- 전달 받은 sleep 값 만큼 대기한다.
- sleep 값을 반환한다.

## ExecutorService - invokeAll()

invokeAll() 은 한 번에 여러 작업을 제출하고, 모든 작업이 완료될 때 까지 기다린다.

```
package thread.executor.future;

import thread.executor.CallableTask;

import java.util.List;
import java.util.concurrent.*;

import static util.MyLogger.log;

public class InvokeAllMain {

    public static void main(String[] args) throws ExecutionException,
        InterruptedException {
        ExecutorService es = Executors.newFixedThreadPool(10);

        CallableTask task1 = new CallableTask("task1", 1000);
        CallableTask task2 = new CallableTask("task2", 2000);
        CallableTask task3 = new CallableTask("task3", 3000);
        List<CallableTask> tasks = List.of(task1, task2, task3);

        List<Future<Integer>> futures = es.invokeAll(tasks);
        for (Future<Integer> future : futures) {
            Integer value = future.get();
            log("value = " + value);
        }
        es.close();
    }
}
```

```
15:42:40.470 [pool-1-thread-1] task1 실행
15:42:40.470 [pool-1-thread-2] task2 실행
15:42:40.470 [pool-1-thread-3] task3 실행
15:42:41.491 [pool-1-thread-1] task1 완료, return = 1000
15:42:42.474 [pool-1-thread-2] task2 완료, return = 2000
15:42:43.473 [pool-1-thread-3] task3 완료, return = 3000
15:42:43.474 [      main] value = 1000
15:42:43.474 [      main] value = 2000
15:42:43.474 [      main] value = 3000
```

## ExecutorService - invokeAny()

`invokeAny()` 는 한 번에 여러 작업을 제출하고, 가장 먼저 완료된 작업의 결과를 반환한다. 이때 완료되지 않은 나머지 작업은 인터럽트를 통해 취소한다.

```
package thread.executor.future;

import thread.executor.CallableTask;

import java.util.List;
import java.util.concurrent.*;

import static util.MyLogger.log;

public class InvokeAnyMain {

    public static void main(String[] args) throws ExecutionException,
        InterruptedException {
        ExecutorService es = Executors.newFixedThreadPool(10);

        CallableTask task1 = new CallableTask("task1", 1000);
        CallableTask task2 = new CallableTask("task2", 2000);
        CallableTask task3 = new CallableTask("task3", 3000);
        List<CallableTask> tasks = List.of(task1, task2, task3);

        Integer value = es.invokeAny(tasks);
        log("value = " + value);
        es.close();
    }
}
```

```
}
```

## 실행 결과

```
15:46:11.722 [pool-1-thread-1] task1 실행
15:46:11.722 [pool-1-thread-2] task2 실행
15:46:11.722 [pool-1-thread-3] task3 실행
15:46:12.741 [pool-1-thread-1] task1 완료, return = 1000
15:46:12.742 [      main] value = 1000
15:46:12.742 [pool-1-thread-2] 인터럽트 발생, sleep interrupted
15:46:12.742 [pool-1-thread-3] 인터럽트 발생, sleep interrupted
```

## 문제와 풀이

당신은 커머스 회사의 주문 팀에 새로 입사했다.

주문 팀의 고민은 연동하는 시스템이 점점 많아지면서 주문 프로세스가 너무 오래 걸린다는 점이다.

하나의 주문이 발생하면 추가로 3가지 일이 발생한다.

- 재고를 업데이트 해야한다. 약 1초
- 배송 시스템에 알려야 한다. 약 1초
- 회계 시스템에 내용을 업데이트 해야 한다. 약 1초

각각 1초가 걸리기 때문에, 고객 입장에서는 보통 3초의 시간을 대기해야 한다.

3가지 업무의 호출 순서는 상관이 없다. 각각에 주문 번호만 잘 전달하면 된다. 물론 3가지 일이 모두 성공해야 주문이 완료된다.

여기에 기존 코드가 있다. 기존 코드를 개선해서 주문 시간을 최대한 줄여보자.

```
package thread.executor.test;

import static util.MyLogger.log;
import static util.ThreadUtils.sleep;

public class OldOrderService {
```

```

public void order(String orderNo) {
    InventoryWork inventoryWork = new InventoryWork(orderNo);
    ShippingWork shippingWork = new ShippingWork(orderNo);
    AccountingWork accountingWork = new AccountingWork(orderNo);

    // 작업 요청
    Boolean inventoryResult = inventoryWork.call();
    Boolean shippingResult = shippingWork.call();
    Boolean accountingResult = accountingWork.call();

    // 결과 확인
    if (inventoryResult && shippingResult && accountingResult) {
        log("모든 주문 처리가 성공적으로 완료되었습니다.");
    } else {
        log("일부 작업이 실패했습니다.");
    }
}

static class InventoryWork {

    private final String orderNo;

    public InventoryWork(String orderNo) {
        this.orderNo = orderNo;
    }

    public Boolean call() {
        log("재고 업데이트: " + orderNo);
        sleep(1000);
        return true;
    }
}

static class ShippingWork {

    private final String orderNo;

    public ShippingWork(String orderNo) {
        this.orderNo = orderNo;
    }

    public Boolean call() {
        log("배송 시스템 알림: " + orderNo);
    }
}

```

```

        sleep(1000);
        return true;
    }
}

static class AccountingWork {

    private final String orderNo;

    public AccountingWork(String orderNo) {
        this.orderNo = orderNo;
    }

    public Boolean call() {
        log("회계 시스템 업데이트: " + orderNo);
        sleep(1000);
        return true;
    }
}
}

```

```

package thread.executor.test;

public class OldOrderServiceTestMain {

    public static void main(String[] args) {
        String orderNo = "Order#1234"; // 예시 주문 번호
        OldOrderService orderService = new OldOrderService();
        orderService.order(orderNo);
    }
}

```

## 실행 결과

```

12:14:21.167 [      main] 재고 업데이트: Order#1234
12:14:22.170 [      main] 배송 시스템 알림: Order#1234
12:14:23.176 [      main] 회계 시스템 업데이트: Order#1234
12:14:24.181 [      main] 모든 주문 처리가 성공적으로 완료되었습니다.

```

총 실행 시간: 3초

정답

```
package thread.executor.test;

import java.util.concurrent.*;

import static util.MyLogger.log;
import static util.ThreadUtils.sleep;

public class NewOrderService {

    private final ExecutorService es = Executors.newFixedThreadPool(10);

    public void order(String orderNo) throws ExecutionException,
    InterruptedException {
        InventoryWork inventoryWork = new InventoryWork(orderNo);
        ShippingWork shippingWork = new ShippingWork(orderNo);
        AccountingWork accountingWork = new AccountingWork(orderNo);

        try {
            // 작업들을 ExecutorService에 제출
            Future<Boolean> inventoryFuture = es.submit(inventoryWork);
            Future<Boolean> shippingFuture = es.submit(shippingWork);
            Future<Boolean> accountingFuture = es.submit(accountingWork);

            // 작업 완료를 기다림
            Boolean inventoryResult = inventoryFuture.get();
            Boolean shippingResult = shippingFuture.get();
            Boolean accountingResult = accountingFuture.get();

            // 결과 확인
            if (inventoryResult && shippingResult && accountingResult) {
                log("모든 주문 처리가 성공적으로 완료되었습니다.");
            } else {
                log("일부 작업이 실패했습니다.");
            }
        } finally {
```

```

        es.close();
    }
}

static class InventoryWork implements Callable<Boolean> {

    private final String orderNo;

    public InventoryWork(String orderNo) {
        this.orderNo = orderNo;
    }

    @Override
    public Boolean call() {
        log("재고 업데이트: " + orderNo);
        sleep(1000);
        return true;
    }
}

static class ShippingWork implements Callable<Boolean> {

    private final String orderNo;

    public ShippingWork(String orderNo) {
        this.orderNo = orderNo;
    }

    @Override
    public Boolean call() {
        log("배송 시스템 알림: " + orderNo);
        sleep(1000);
        return true;
    }
}

static class AccountingWork implements Callable<Boolean> {

    private final String orderNo;

    public AccountingWork(String orderNo) {
        this.orderNo = orderNo;
    }
}

```

```

@Override
public Boolean call() {
    log("회계 시스템 업데이트: " + orderNo);
    sleep(1000);
    return true;
}
}
}

```

- `ExecutorService` 를 사용했다.
- 기존 코드에서 `Callable`, `ExecutorService` 를 사용하도록 약간만 변경하면 된다.
- 3가지 업무는 서로 순서상 관계가 없으므로 스레드를 나누어 함께 실행하고 그 결과만 확인하면 된다.
- 예제를 단순화 하기 위해 예외는 처리하지 않았다.
- 물론 `invokeAll()` 과 같은 기능을 사용해서 풀어도 된다.

```

package thread.executor.test;

import java.util.concurrent.*;

public class NewOrderServiceTestMain {

    public static void main(String[] args) throws ExecutionException,
    InterruptedException {
        String orderNo = "Order#1234"; // 예시 주문 번호
        NewOrderService orderService = new NewOrderService();
        orderService.order(orderNo);
    }
}

```

## 실행 결과

```

12:15:06.315 [pool-1-thread-1] 재고 업데이트: Order#1234
12:15:06.315 [pool-1-thread-2] 배송 시스템 알림: Order#1234
12:15:06.315 [pool-1-thread-3] 회계 시스템 업데이트: Order#1234
12:15:07.323 [      main] 모든 주문 처리가 성공적으로 완료되었습니다.

```

총 실행 시간: 1초

# 정리

ExecutorService 를 자세히 정리해보자.

## Executor 인터페이스

```
package java.util.concurrent;

public interface Executor {
    void execute(Runnable command);
}
```

- 가장 단순한 작업 실행 인터페이스로, execute(Runnable command) 메서드 하나를 가지고 있다.

## ExecutorService 인터페이스 - 주요 메서드

```
public interface ExecutorService extends Executor, AutoCloseable {

    // 종료 메서드
    void shutdown();
    List<Runnable> shutdownNow();
    boolean isShutdown();
    boolean isTerminated();
    boolean awaitTermination(long timeout, TimeUnit unit)
        throws InterruptedException;

    // 단일 실행
    <T> Future<T> submit(Callable<T> task);
    <T> Future<T> submit(Runnable task, T result);
    Future<?> submit(Runnable task);

    // 다수 실행
    <T> List<Future<T>> invokeAll(Collection<? extends Callable<T>> tasks)
        throws InterruptedException;
    <T> List<Future<T>> invokeAll(Collection<? extends Callable<T>> tasks,
                                long timeout, TimeUnit unit)
        throws InterruptedException;
```

```

<T> T invokeAny(Collection<? extends Callable<T>> tasks)
    throws InterruptedException, ExecutionException;
<T> T invokeAny(Collection<? extends Callable<T>> tasks,
    long timeout, TimeUnit unit)
    throws InterruptedException, ExecutionException, TimeoutException;

@Override
default void close(){...}
}

```

- `Executor` 인터페이스를 확장하여 작업 제출과 제어 기능을 추가로 제공한다.
- 주요 메서드로는 `submit()`, `invokeAll()`, `invokeAny()`, `shutdown()` 등이 있다.
- `Executor` 프레임워크를 사용할 때는 대부분 이 인터페이스를 사용한다.
- `ExecutorService` 인터페이스의 기본 구현체는 `ThreadPoolExecutor` 이다.

## ExecutorService 주요 메서드 정리

### 작업 제출 및 실행

- `void execute(Runnable command): Runnable` 작업을 제출한다. 반환값이 없다.
- `<T> Future<T> submit(Callable<T> task): Callable` 작업을 제출하고 결과를 반환받는다.
- `Future<?> submit(Runnable task): Runnable` 작업을 제출하고 결과를 반환받는다.

`ExecutorService.submit()`에는 반환 결과가 있는 `Callable` 뿐만 아니라 반환 결과가 없는 `Runnable`도 사용할 수 있다.

예를 들어 다음 코드도 가능하다.

```
Future<?> future = executor.submit(new MyRunnable());
```

`Runnable`은 반환 값이 없기 때문에 `future.get()`을 호출할 경우 `null`을 반환한다. 결과가 없다 뿐이지 나머지는 똑같다. 작업이 완료될 때 까지 요청 스레드가 블로킹 되는 부분도 같다.

### ExecutorService 종료

자바 19부터 `close()`가 제공된다. `shutdown()`을 포함한 `ExecutorService` 종료에 대한 부분은 뒤에서 자세히 다룬다.

### 작업 컬렉션 처리

## invokeAll()

- `<T> List<Future<T>> invokeAll(Collection<? extends Callable<T>> tasks) throws InterruptedException`
  - 모든 `Callable` 작업을 제출하고, 모든 작업이 완료될 때까지 기다린다.
- `<T> List<Future<T>> invokeAll(Collection<? extends Callable<T>> tasks, long timeout, TimeUnit unit) throws InterruptedException`
  - 지정된 시간 내에 모든 `Callable` 작업을 제출하고 완료될 때까지 기다린다.

## invokeAny()

- `<T> T invokeAny(Collection<? extends Callable<T>> tasks) throws InterruptedException, ExecutionException`
  - 하나의 `Callable` 작업이 완료될 때까지 기다리고, 가장 먼저 완료된 작업의 결과를 반환한다.
  - 완료되지 않은 나머지 작업은 취소한다.
- `<T> T invokeAny(Collection<? extends Callable<T>> tasks, long timeout, TimeUnit unit) throws InterruptedException, ExecutionException, TimeoutException`
  - 지정된 시간 내에 하나의 `Callable` 작업이 완료될 때까지 기다리고, 가장 먼저 완료된 작업의 결과를 반환한다.
  - 완료되지 않은 나머지 작업은 취소한다.